

README

Clay, A UI Layout Library

Clay (short for **C Layout**) is a high performance 2D UI layout library.

Major Features

- Microsecond layout performance
- Flex-box like layout model for complex, responsive layouts including text wrapping, scrolling containers and aspect ratio scaling
- Single ~4k LOC **clay.h** file with **zero** dependencies (including no standard library)
- Wasm support: compile with clang to a 15kb uncompressed **.wasm** file for use in the browser
- Static arena based memory use with no malloc / free, and low total memory overhead (e.g. ~3.5mb for 8192 layout elements).
- React-like nested declarative syntax
- Renderer agnostic: outputs a sorted list of rendering primitives that can be easily composited in any 3D engine, and even compiled to HTML (examples provided)

Take a look at the [clay website](#) for an example of clay compiled to wasm and running in the browser, or others in the [examples directory](#).

You can also watch the [introduction video](#) for an overview of the motivation behind Clay's development and a short demo of its usage.

```
1 header.inc()
```

```
Begin CODE segment (8000)
```

```
Reset: Reset handler (called when the NES resets or powers on)
```

```
2 INIT_NES() Macro to initialize the NES to a known state
```

```
Main:
```

```
3 BIT m PPU_STATUS + Read PPU_STATUS to reset the PPU_ADDR latch
```

```
4 LDX x3F
```

```
5 STX m PPU_ADDR + Set hi-byte of PPU_ADDR to $3F
```

```
6 LDX x0
```

```
7 STX m PPU_ADDR + Set lo-byte of PPU_ADDR to $00
```

```
8 LoadPalette() Jump to subroutine LoadPalette
```

```
9 LDY d0 Y = 0
```

```
-
```

```
10 LDA m PaletteData + Y Lookup byte in ROM
```

```
11 STA m PPU_DATA + Set value to send to PPU_DATA
```

```
12 INY Y++
```

```
13 CPY d32 Is Y equal to 32?
```

```
14 BNE m - + Not yet, keep looping
```

```
15 BIT m PPU_STATUS + Read PPU_STATUS to reset the PPU_ADDR latch
```

```
16 LDX x20
```

```
17 STX m PPU_ADDR + Set hi-byte of PPU_ADDR to $20
```

```
18 LDX x0
```

```
19 STX m PPU_ADDR + Set lo-byte of PPU_ADDR to $00
```

```
20 LoadBackground() Jump to subroutine LoadBackground
```

```
21 BIT m PPU_STATUS + Nametable Attributes  
Read PPU_STATUS to reset the PPU_ADDR
```

```
22 LDX x23 latch
```

```
23 STX m PPU_ADDR + Set hi-byte of PPU_ADDR to $23
```

Global Var

Frame

address
0 Referen

Clock60

address
0 Referen

Background

address
0 Referen

Quick Start

Download or clone clay.h and include it after defining `CLAY_IMPLEMENTATION` in one file.

```
// Must be defined in one file, _before_ #include "clay.h"
#define CLAY_IMPLEMENTATION
#include "../.. /clay.h"

const Clay_Color COLOR_LIGHT = (Clay_Color) {224, 215, 210, 255};
const Clay_Color COLOR_RED = (Clay_Color) {168, 66, 28, 255};
const Clay_Color COLOR_ORANGE = (Clay_Color) {225, 138, 50, 255};

void HandleClayErrors(Clay_ErrorData errorData) {
    // See the Clay_ErrorData struct for more information
    printf("%s", errorData.errorText.chars);
    switch(errorData.errorType) {
        // etc
    }
}

// Example measure text function
static inline Clay_Dimensions MeasureText(Clay_StringSlice text, Clay_TextElementConfig *config, uintptr_t
userData) {
    // Clay_TextElementConfig contains members such as fontId, fontSize, letterSpacing etc
    // Note: Clay_String->chars is not guaranteed to be null terminated
    return (Clay_Dimensions) {
        .width = text.length * config->fontSize, // <- this will only work for monospace fonts, see the
renderers/ directory for more advanced text measurement
        .height = config->fontSize
    };
}

// Layout config is just a struct that can be declared statically, or inline
Clay_ElementDeclaration sidebarItemConfig = (Clay_ElementDeclaration) {
    .layout = {
```

```

        .sizing = { .width = CLAY_SIZING_GROW(0), .height = CLAY_SIZING_FIXED(50) }
    },
    .backgroundColor = COLOR_ORANGE
};

// Re-useable components are just normal functions
void SidebarItemComponent() {
    CLAY(id, sidebarItemConfig) {
        // children go here...
    }
}

int main() {
    // Note: malloc is only used here as an example, any allocator that provides
    // a pointer to addressable memory of at least totalMemorySize will work
    uint64_t totalMemorySize = Clay_MinMemorySize();
    Clay_Arena arena = Clay_CreateArenaWithCapacityAndMemory(totalMemorySize, malloc(totalMemorySize));

    // Note: screenWidth and screenHeight will need to come from your environment, Clay doesn't handle window
    // related tasks
    Clay_Initialize(arena, (Clay_Dimensions) { screenWidth, screenHeight }, (Clay_ErrorHandler) { HandleClayErrors
});

    while(renderLoop()) { // Will be different for each renderer / environment
        // Optional: Update internal layout dimensions to support resizing
        Clay_SetLayoutDimensions((Clay_Dimensions) { screenWidth, screenHeight });
        // Optional: Update internal pointer position for handling mouseover / click / touch events - needed for
        // scrolling & debug tools
        Clay_SetPointerState((Clay_Vector2) { mouseX, mouseY }, isMouseDown);
        // Optional: Update internal pointer position for handling mouseover / click / touch events - needed for
        // scrolling and debug tools
        Clay_UpdateScrollContainers(true, (Clay_Vector2) { mouseX, mouseY }, deltaTime);

        // All clay layouts are declared between Clay_BeginLayout and Clay_EndLayout
        Clay_BeginLayout();

        // An example of laying out a UI with a fixed width sidebar and flexible width main content
        CLAY(CLAY_ID("OuterContainer"), { .layout = { .sizing = {CLAY_SIZING_GROW(0), CLAY_SIZING_GROW(0)},

```

```

.padding = CLAY_PADDING_ALL(16), .childGap = 16 }, .backgroundColor = {250,250,255,255} }) {
    CLAY(CLAY_ID("SideBar"), {
        .layout = { .layoutDirection = CLAY_TOP_TO_BOTTOM, .sizing = { .width = CLAY_SIZING_FIXED(300),
.height = CLAY_SIZING_GROW(0) }, .padding = CLAY_PADDING_ALL(16), .childGap = 16 },
        .backgroundColor = COLOR_LIGHT
    }) {
        CLAY(CLAY_ID("ProfilePictureOuter"), { .layout = { .sizing = { .width = CLAY_SIZING_GROW(0) },
.padding = CLAY_PADDING_ALL(16), .childGap = 16, .childAlignment = { .y = CLAY_ALIGN_Y_CENTER } },
.backgroundColor = COLOR_RED }) {
            CLAY(CLAY_ID("ProfilePicture"), { .layout = { .sizing = { .width = CLAY_SIZING_FIXED(60),
.height = CLAY_SIZING_FIXED(60) } }, .image = { .imageData = &profilePicture } }) {}
            CLAY_TEXT(CLAY_STRING("Clay - UI Library"), CLAY_TEXT_CONFIG({ .fontSize = 24, .textColor =
{255, 255, 255, 255} }));
        }

        // Standard C code like loops etc work inside components
        for (int i = 0; i < 5; i++) {
            SidebarItemComponent();
        }

        CLAY(CLAY_ID("MainContent"), { .layout = { .sizing = { .width = CLAY_SIZING_GROW(0), .height =
CLAY_SIZING_GROW(0) } }, .backgroundColor = COLOR_LIGHT }) {}
    }
}

// All clay layouts are declared between Clay_BeginLayout and Clay_EndLayout
Clay_RenderCommandArray renderCommands = Clay_EndLayout();

// More comprehensive rendering examples can be found in the renderers/ directory
for (int i = 0; i < renderCommands.length; i++) {
    Clay_RenderCommand *renderCommand = &renderCommands.internalArray[i];

    switch (renderCommand->commandType) {
        case CLAY_RENDER_COMMAND_TYPE_RECTANGLE: {
            DrawRectangle( renderCommand->boundingBox, renderCommand-
>renderData.rectangle.backgroundColor);
        }
        // ... Implement handling of other command types
    }
}

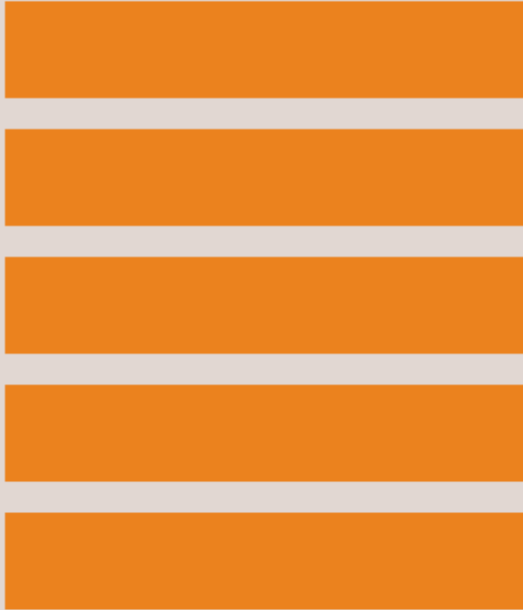
```

```
}  
  }  
    }  
      }
```

The above example, rendered correctly will look something like the following:



Clay - UI Library



In summary, the general order of steps is:

1. `Clay_SetLayoutDimensions(dimensions)`
2. `Clay_SetPointerState(pointerPosition, isPointerDown)`
3. `Clay_UpdateScrollContainers(enableDragScrolling, scrollDelta, deltaTime)`
4. `Clay_BeginLayout()`
5. Declare your layout with the provided [Element Macros](#)
6. `Clay_EndLayout()`
7. Render the results using the outputted `Clay_RenderCommandArray`

For help starting out or to discuss clay, considering joining [the discord server](#).

Summary

- High Level Documentation
 - [Building UI Hierarchies](#)
 - [Configuring Layout and Styling UI Elements](#)
 - [Element IDs](#)
 - [Mouse, Touch and Pointer Interactions](#)
 - [Scrolling Elements](#)
 - [Floating Elements](#)
 - [Custom Elements](#)
 - [Retained Mode Rendering](#)
 - [Visibility Culling](#)
 - [Preprocessor Directives](#)
 - [Bindings](#)
 - [Debug Tools](#)
- API
 - [Naming Conventions](#)
 - [Public Functions](#)
 - [Lifecycle](#)
 - [Clay_MinMemorySize](#)
 - [Clay_CreateArenaWithCapacityAndMemory](#)
 - [Clay_SetMeasureTextFunction](#)
 - [Clay_ResetMeasureTextCache](#)
 - [Clay_SetMaxElementCount](#)

- Clay_SetMaxMeasureTextCacheWordCount
- Clay_Initialize
- Clay_GetCurrentContext
- Clay_SetCurrentContext
- Clay_SetLayoutDimensions
- Clay_SetPointerState
- Clay_UpdateScrollContainers
- Clay_BeginLayout
- Clay_EndLayout
- Clay_Hovered
- Clay_OnHover
- Clay_PointerOver
- Clay_GetScrollContainerData
- Clay_GetElementData
- Clay_GetElementId
- Element Macros
 - CLAY
 - CLAY_ID
 - CLAY_IDI
- Data Structures & Defs
 - Clay_String
 - Clay_ElementId
 - Clay_RenderCommandArray
 - Clay_RenderCommand
 - Clay_ScrollContainerData
 - Clay_ErrorHandler
 - Clay_ErrorData

High Level Documentation

Building UI Hierarchies

Clay UIs are built using the C macro `CLAY(id, { configuration })`. This macro creates a new empty element in the UI hierarchy, and supports modular customisation of layout, styling and functionality. The `CLAY()` macro can also be *nested*, similar to other declarative UI

systems like HTML.

Child elements are added by opening a block: `{ }` after calling the `CLAY()` macro (exactly like you would with an `if` statement or `for` loop), and declaring child components inside the braces.

```
// Parent element with 8px of padding
CLAY(CLAY_ID("parent"), { .layout = { .padding = CLAY_PADDING_ALL(8) } }) {
    // Child element 1
    CLAY_TEXT(CLAY_STRING("Hello World"), CLAY_TEXT_CONFIG({ .fontSize = 16 }));
    // Child element 2 with red background
    CLAY((CLAY_ID("child"), { .backgroundColor = COLOR_RED }) {
        // etc
    }
}
```

However, unlike HTML and other declarative DSLs, this macro is just C. As a result, you can use arbitrary C code such as loops, functions and conditions inside your layout declaration code:

```
// Re-usable "components" are just functions that declare more UI
void ButtonComponent(Clay_String buttonText) {
    // Red box button with 8px of padding
    CLAY_AUTO_ID({ .layout = { .padding = CLAY_PADDING_ALL(8) }, .backgroundColor = COLOR_RED }) {
        CLAY_TEXT(buttonText, textConfig);
    }
}

// Parent element
CLAY(CLAY_ID("parent"), { .layout = { .layoutDirection = CLAY_TOP_TO_BOTTOM } }) {
    // Render a bunch of text elements
    for (int i = 0; i < textArray.length; i++) {
        CLAY_TEXT(textArray.elements[i], textConfig);
    }
    // Only render this element if we're on a mobile screen
    if (isMobileScreen) {
        CLAY(0) {
            // etc
        }
    }
}
```

```

    }
    // Re-usable components
    ButtonComponent(CLAY_STRING("Click me!"));
    ButtonComponent(CLAY_STRING("No, click me!"));
});

```

Configuring Layout and Styling UI Elements

The layout and style of clay elements is configured with the [Clay_ElementDeclaration](#) struct passed to the `CLAY()` macro.

```

CLAY(CLAY_ID("box"), { .layout = { .padding = { 8, 8, 8, 8 }, .layoutDirection = CLAY_TOP_TO_BOTTOM } }) {
    // Children are 8px inset into parent, and laid out top to bottom
}

```

This macro isn't magic - all it's doing is wrapping the standard designated initializer syntax. e.g. `(Clay_ElementDeclaration) { .layout = { .padding = { .left = 8, .right = 8 } ... }`.

See the [Clay_ElementDeclaration](#) API for the full list of options.

A `Clay_ElementDeclaration` struct can be defined in file scope or elsewhere, and reused.

```

// Define a style in the global / file scope
Clay_ElementDeclaration reuseableStyle = (Clay_ElementDeclaration) {
    .layout = { .padding = { .left = 12 } },
    .backgroundColor = { 120, 120, 120, 255 },
    .cornerRadius = { 12, 12, 12, 12 }
};

CLAY(CLAY_ID("box"), reuseableStyle) {
    // ...
}

```

Element IDs

The Clay macro by default accepts an ID as its first argument, which is usually provided by the [CLAY_ID\(\)](#) convenience macro. Elements can also be created with auto generated IDs, by using the [CLAY_AUTO_ID\(\)](#) macro.

```
// Will always produce the same ID from the same input string
CLAY(CLAY_ID("OuterContainer"), { ...configuration }) {}

// Generates a unique ID that may not be the same between two layout calls
CLAY_AUTO_ID({ ...configuration }) {}
```

Element IDs have two main use cases. Firstly, tagging an element with an ID allows you to query information about the element later, such as its [mouseover state](#) or dimensions.

Secondly, IDs are visually useful when attempting to read and modify UI code, as well as when using the built-in [debug tools](#).

To avoid having to construct dynamic strings at runtime to differentiate ids in loops, clay provides the [CLAY_IDI\(string, index\)](#) macro to generate different ids from a single input string. Think of IDI as "[ID](#) + [Index](#)"

```
// This is the equivalent of calling CLAY_ID("Item0"), CLAY_ID("Item1") etc
for (int index = 0; index < items.length; index++) {
    CLAY(CLAY_IDI("Item", index), { ..configuration }) {}
}
```

This ID will be forwarded to the final `Clay_RenderCommandArray` for use in retained mode UIs. Using duplicate IDs may cause some functionality to misbehave (i.e. if you're trying to attach a floating container to a specific element with ID that is duplicated, it may not attach to the one you expect)

Mouse, Touch and Pointer Interactions

Clay provides several functions for handling mouse and pointer interactions.

All pointer interactions depend on the function `void Clay_SetPointerState(Clay_Vector2 position, bool isPointerDown)` being called after each mouse position update and before any other clay functions.

During UI declaration

The function `bool Clay_Hovered()` can be called during element construction or in the body of an element, and returns `true` if the mouse / pointer is over the currently open element.

```
// An orange button that turns blue when hovered
CLAY(CLAY_ID("Button"), { .backgroundColor = Clay_Hovered() ? COLOR_BLUE : COLOR_ORANGE }) {
    bool buttonHovered = Clay_Hovered();
    CLAY_TEXT(buttonHovered ? CLAY_STRING("Hovered") : CLAY_STRING("Hover me!"), headerTextConfig);
}
```

The function `void Clay_OnHover()` allows you to attach a function pointer to the currently open element, which will be called if the mouse / pointer is over the element.

```
void HandleButtonInteraction(Clay_ElementId elementId, Clay_PointerData pointerInfo, void *userData) {
    ButtonData *buttonData = (ButtonData *)userData;
    // Pointer state allows you to detect mouse down / hold / release
    if (pointerInfo.state == CLAY_POINTER_DATA_PRESSED_THIS_FRAME) {
        // Do some click handling
        NavigateTo(buttonData->link);
    }
}

ButtonData linkButton = (ButtonData) { .link = "https://github.com/nicbarker/clay" };

// HandleButtonInteraction will be called for each frame the mouse / pointer / touch is inside the button
// boundaries
CLAY(CLAY_ID("Button"), { .layout = { .padding = CLAY_PADDING_ALL(8) } }) {
    Clay_OnHover(HandleButtonInteraction, &linkButton);
    CLAY_TEXT(CLAY_STRING("Button"), &headerTextConfig);
}
```

Before / After UI declaration

If you want to query mouse / pointer overlaps outside layout declarations, you can use the function `bool Clay_PointerOver(Clay_ElementId id)`, which takes an `element id` and returns a bool representing whether the current pointer position is within its bounding box.

```
// Reminder: Clay_SetPointerState must be called before functions that rely on pointer position otherwise it will
// have no effect
Clay_Vector2 mousePosition = { x, y };
```

```

Clay_SetPointerType(mousePosition);
// ...
// If profile picture was clicked
if (mouseButtonDown(0) && Clay_PointerOver(Clay_GetElementId("ProfilePicture"))) {
    // Handle profile picture clicked
}

```

Note that the bounding box queried by `Clay_PointerOver` is from the last frame. This generally shouldn't make a difference except in the case of animations that move at high speed.

If this is an issue for you, performing layout twice per frame with the same data will give you the correct interaction the second time.

Scrolling Elements

Elements are configured as scrollable with the `.clip` configuration. Clipping instructs the renderer to not draw any pixels outside the clipped element's boundaries, and by specifying the `.childOffset` field, the clipped element's contents can be shifted around to provide "scrolling" behaviour.

You can either calculate scrolling yourself and simply provide the current offset each frame to `.childOffset`, or alternatively, Clay provides a built in mechanism for tracking and updating scroll container offsets, detailed below.

To make scroll containers respond to mouse wheel and scroll events, two functions need to be called before `BeginLayout()`:

```

Clay_Vector2 mousePosition = { x, y };
// Reminder: Clay_SetPointerType must be called before Clay_UpdateScrollContainers otherwise it will have no effect
Clay_SetPointerType(mousePosition);
// Clay_UpdateScrollContainers needs to be called before Clay_BeginLayout for the position to avoid a 1 frame delay
Clay_UpdateScrollContainers(
    true, // Enable drag scrolling
    scrollDelta, // Clay_Vector2 scrollwheel / trackpad scroll x and y delta this frame
    float deltaTime, // Time since last frame in seconds as a float e.g. 8ms is 0.008f
);
// ...
// Clay internally tracks the scroll containers offset, and Clay_GetScrollOffset returns the x,y offset of the currently open element
CLAY(CLAY_ID("ScrollContainer"), { .clip = { .vertical = true, .childOffset = Clay_GetScrollOffset() } }) {

```

```
// Scrolling contents
}
// .childOffset can be provided directly if you would prefer to manage scrolling outside of clay
CLAY(CLAY_ID("ScrollContainer"), { .clip = { .vertical = true, .childOffset = myData.scrollContainer.offset } }) {
    // Scrolling contents
}
```

More specific details can be found in the docs for [Clay_UpdateScrollContainers](#), [Clay_SetPointerState](#), [Clay_ClipElementConfig](#) and [Clay_GetScrollOffset](#).

Floating Elements ("Absolute" Positioning)

All standard elements in clay are laid out on top of, and *within* their parent, positioned according to their parent's layout rules, and affect the positioning and sizing of siblings.

"**Floating**" is configured with the `CLAY_FLOATING()` macro. Floating elements don't affect the parent they are defined in, or the position of their siblings.

They also have a **z-index**, and as a result can intersect and render over the top of other elements.

A classic example use case for floating elements is tooltips and modals.

```
// The two text elements will be laid out top to bottom, and the floating container
// will be attached to "Outer"
CLAY(CLAY_ID("Outer"), { .layout = { .layoutDirection = TOP_TO_BOTTOM } }) {
    CLAY_TEXT(text, &headerTextConfig);
    CLAY(CLAY_ID("Tooltip"), { .floating = { .attachTo = CLAY_ATTACH_TO_PARENT } }) {}
    CLAY_TEXT(text, &headerTextConfig);
}
```

More specific details can be found in the full [Floating API](#).

Laying Out Your Own Custom Elements

Clay only supports a simple set of UI element primitives, such as rectangles, text and images. Clay provides a singular API for layout out custom elements:

```

#include "clay.h"

typedef enum {
    CUSTOM_ELEMENT_TYPE_MODEL,
    CUSTOM_ELEMENT_TYPE_VIDEO
} CustomElementType;

// A rough example of how you could handle laying out 3d models in your UI
typedef struct {
    CustomElementType type;
    union {
        Model model;
        Video video;
        // ...
    };
} CustomElementData;

Model myModel = Load3DModel(filePath);
CustomElement modelElement = (CustomElement) { .type = CUSTOM_ELEMENT_TYPE_MODEL, .model = myModel }

typedef struct {
    void* memory;
    uintptr_t offset;
} Arena;

// During init
Arena frameArena = (Arena) { .memory = malloc(1024) };

// Custom elements only take a single pointer, so we need to store the data somewhere
CustomElementData *modelData = (CustomElementData *) (frameArena.memory + frameArena.offset);
*modelData = (CustomElementData) { .type = CUSTOM_ELEMENT_TYPE_MODEL, .model = myModel };
frameArena.offset += sizeof(CustomElementData);
CLAY(CLAY_ID("3DModelViewer"), { .custom = { .customData = modelData } }) {}

// Later during your rendering
switch (renderCommand->commandType) {
    // ...
    case CLAY_RENDER_COMMAND_TYPE_CUSTOM: {

```



```

// Your extended struct is passed through
CustomElementData *customElement = renderCommand->config.customElementConfig->customData;
if (!customElement) continue;
switch (customElement->type) {
    case CUSTOM_ELEMENT_TYPE_MODEL: {
        // Render your 3d model here
        break;
    }
    case CUSTOM_ELEMENT_TYPE_VIDEO: {
        // Render your video here
        break;
    }
    // ...
}
break;
}
}
}

```

More specific details can be found in the full [Custom Element API](#).

Retained Mode Rendering

Clay was originally designed for [Immediate Mode](#) rendering - where the entire UI is redrawn every frame. This may not be possible with your platform, renderer design or performance constraints.

There are some general techniques that can be used to integrate clay into a retained mode rendering system:

- `Clay_RenderCommand` includes the `uint32_t id` that was used to declare the element. If unique ids are used, these can be mapped to persistent graphics objects across multiple frames / layouts.
- Render commands are culled automatically to only currently visible elements, and `Clay_RenderCommand` is a small enough struct that you can simply compare the memory of two render commands with matching IDs to determine if the element is "dirty" and needs to be re-rendered or updated.

For a worked example, see the provided [HTML renderer](#). This renderer converts clay layouts into persistent HTML documents with minimal changes per frame.

Visibility Culling

Clay provides a built-in visibility-culling mechanism that is **enabled by default**. It will only output render commands for elements that are visible - that is, **at least one pixel of their bounding box is inside the viewport**.

This culling mechanism can be disabled via the use of the `#define CLAY_DISABLE_CULLING` directive. See [Preprocessor Directives](#) for more information.

Preprocessor Directives

Clay supports C preprocessor directives to modulate functionality at compile time. These can be set either in code using `#define CLAY_DISABLE_CULLING` or on the command line when compiling using the appropriate compiler specific arguments, e.g. `clang -DCLAY_DISABLE_CULLING main.c ...`

The supported directives are:

- `CLAY_WASM` - Required when targeting Web Assembly.
- `CLAY_DLL` - Required when creating a .Dll file.

Bindings for non C

Clay is usable out of the box as a `.h` include in both C99 and C++20 with designated initializer support.

There are also supported bindings for other languages, including:

- [Odin Bindings](#)
- [Rust Bindings](#)

Other implementations

Clay has also been implemented in other languages:

- `glay` - Go line-by-line rewrite with readability as main goal.
- `totallygamerjet/clay` - Port using `cxgo`, a C to Go transpiler.
- `goclay` - Go line-by-line rewrite closely matching the reference.

Debug Tools

Clay includes built-in UI debugging tools, similar to the "inspector" in browsers such as Chrome or Firefox. These tools are included in `clay.h`, and work by injecting additional render commands into the output `Clay_RenderCommandArray`.

As long as the renderer that you're using works correctly, no additional setup or configuration is required to use the debug tools.

To enable the debug tools, use the function `Clay_SetDebugModeEnabled(bool enabled)`. This boolean is persistent and does not need to be set every frame.

The debug tooling by default will render as a panel to the right side of the screen, compressing your layout by its width. The default width is 400 and is currently configurable via the direct mutation of the internal variable `Clay__debugViewWidth`, however this is an internal API and is potentially subject to change.

Clay is a flex-box style UI auto layout library in C, with declarative syntax and microsecond performance.

Clay is laying out this webpage right now!



High performance



Flexbox-style responsive layout



Declarative syntax



Single .h file for C/C++



Compile to 15kb .wasm

Running more than one Clay instance

Clay allows you to run more than one instance in a program. To do this, [Clay_Initialize](#) returns a [Clay_Context*](#) reference. You can activate a specific instance using [Clay_SetCurrentContext](#). If [Clay_SetCurrentContext](#) is not called, then Clay will default to using the context from the most recently called [Clay_Initialize](#).

▲ Important: Do not render instances across different threads simultaneously, as Clay does not currently support proper multi-threading.

```
// Define separate arenas for the instances.
Clay_Arena arena1, arena2;
// ... allocate arenas

// Initialize both instances, storing the context for each one.
Clay_Context* instance1 = Clay_Initialize(arena1, layoutDimensions, errorHandler);
Clay_Context* instance2 = Clay_Initialize(arena2, layoutDimensions, errorHandler);

// In the program's render function, activate each instance before executing clay commands and macros.
Clay_SetCurrentContext(instance1);
Clay_BeginLayout();
// ... declare layout for instance1
Clay_RenderCommandArray renderCommands1 = Clay_EndLayout();
render(renderCommands1);

// Switch to the second instance
Clay_SetCurrentContext(instance2);
Clay_BeginLayout();
// ... declare layout for instance2
Clay_RenderCommandArray renderCommands2 = Clay_EndLayout();
render(renderCommands2);
```

API

Naming Conventions

- "CAPITAL_LETTERS()" are used for macros.
- "Clay__" ("Clay" followed by **double** underscore) is used for internal functions that are not intended for use and are subject to change.
- "Clay_" ("Clay" followed by **single** underscore) is used for external functions that can be called by the user.

Public Functions

Lifecycle for public functions

At startup / initialization time, run once

```
Clay_MinMemorySize -> Clay_CreateArenaWithCapacityAndMemory -> Clay_Initialize -> Clay_SetMeasureTextFunction
```

Each Frame

```
Clay_SetLayoutDimensions -> Clay_SetPointerState -> Clay_UpdateScrollContainers -> Clay_BeginLayout -> CLAY() etc... -> Clay_EndLayout
```

Clay_MinMemorySize

```
uint32_t Clay_MinMemorySize()
```

Returns the minimum amount of memory **in bytes** that clay needs to accommodate the current `CLAY_MAX_ELEMENT_COUNT`.

Clay_CreateArenaWithCapacityAndMemory

```
Clay_Arena Clay_CreateArenaWithCapacityAndMemory(uint32_t capacity, void *offset)
```

Creates a `Clay_Arena` struct with the given capacity and base memory pointer, which can be passed to [Clay_Initialize](#).

Clay_SetMeasureTextFunction

```
void Clay_SetMeasureTextFunction(Clay_Dimensions (*measureTextFunction)(Clay_StringSlice text, Clay_TextElementConfig *config, uintptr_t userData), uintptr_t userData)
```

Takes a pointer to a function that can be used to measure the `width, height` dimensions of a string. Used by clay during layout to determine `CLAY_TEXT` element sizing and wrapping.

Note 1: This string is not guaranteed to be null terminated. Clay saves significant performance overhead by using slices when wrapping text instead of having to clone new null terminated strings. If your renderer does not support `ptr, length` style strings (e.g. Raylib), you will need to clone this to a new C string before rendering.

Note 2: It is essential that this function is as fast as possible. For text heavy use-cases this function is called many times, and despite the fact that clay caches text measurements internally, it can easily become the dominant overall layout cost if the provided function is slow. **This is on the hot path!**

Clay_ResetMeasureTextCache

```
void Clay_ResetMeasureTextCache(void)
```

Clay caches measurements from the provided MeasureTextFunction, and this will be sufficient for the majority of use-cases. However, if the measurements can depend on external factors that clay does not know about, like DPI changes, then the cached values may be incorrect. When one of these external factors changes, Clay_ResetMeasureTextCache can be called to force clay to recalculate all string measurements in the next frame.

Clay_SetMaxElementCount

```
void Clay_SetMaxElementCount(uint32_t maxElementCount)
```

Sets the internal maximum element count that will be used in subsequent `Clay_Initialize()` and `Clay_MinMemorySize()` calls, allowing clay to allocate larger UI hierarchies.

Note: You will need to reinitialize clay, after calling `Clay_MinMemorySize()` to calculate updated memory requirements.

Clay_SetMaxMeasureTextCacheWordCount

```
void Clay_SetMaxMeasureTextCacheWordCount(uint32_t maxMeasureTextCacheWordCount)
```

Sets the internal text measurement cache size that will be used in subsequent [Clay_Initialize\(\)](#) and [Clay_MinMemorySize\(\)](#) calls, allowing clay to allocate more text. The value represents how many separate words can be stored in the text measurement cache.

Note: You will need to reinitialize clay, after calling `Clay_MinMemorySize()` to calculate updated memory requirements.

Clay_Initialize

```
Clay_Context* Clay_Initialize(Clay_Arena arena, Clay_Dimensions layoutDimensions, Clay_ErrorHandler errorHandler)
```

Initializes the internal memory mapping, sets the internal dimensions for layout, and binds an error handler for clay to use when something goes wrong. Returns a [Clay_Context*](#) that can optionally be given to [Clay_SetCurrentContext](#) to allow running multiple instances of clay in the same program, and sets it as the current context. See [Running more than one Clay instance](#).

Reference: [Clay_Arena](#), [Clay_ErrorHandler](#), [Clay_SetCurrentContext](#)

Clay_SetCurrentContext

```
void Clay_SetCurrentContext(Clay_Context* context)
```

Sets the context that subsequent clay commands will operate on. You can get this reference from [Clay_Initialize](#) or [Clay_GetCurrentContext](#). See [Running more than one Clay instance](#).

Clay_GetCurrentContext

```
Clay_Context* Clay_GetCurrentContext()
```

Returns the context that clay commands are currently operating on, or null if no context has been set. See [Running more than one Clay instance](#).

Clay_SetLayoutDimensions

```
void Clay_SetLayoutDimensions(Clay_Dimensions dimensions)
```

Sets the internal layout dimensions. Cheap enough to be called every frame with your screen dimensions to automatically respond to window resizing, etc.

Clay_SetPointerState

```
void Clay_SetPointerState(Clay_Vector2 position, bool isPointerDown)
```

Sets the internal pointer position and state (i.e. current mouse / touch position) and recalculates overlap info, which is used for mouseover / click calculation (via [Clay_PointerOver](#) and updating scroll containers with [Clay_UpdateScrollContainers](#). **isPointerDown should represent the current state this frame, e.g. it should be `true` for the entire duration the left mouse button is held down.** Clay has internal handling for detecting click / touch start & end.

Clay_UpdateScrollContainers

```
void Clay_UpdateScrollContainers(bool enableDragScrolling, Clay_Vector2 scrollDelta, float deltaTime)
```

This function handles scrolling of containers. It responds to both `scrollDelta`, which represents mouse wheel or trackpad scrolling this frame, as well as "touch scrolling" on mobile devices, or "drag scrolling" with a mouse or similar device.

Touch / drag scrolling only occurs if the `enableDragScrolling` parameter is `true`, and `Clay_SetPointerState` has been called this frame. As a result, you can simply always call it with `false` as the first argument if you want to disable touch scrolling.

`deltaTime` is the time in seconds since the last frame (e.g. 0.016 is 16 milliseconds), and is used to normalize & smooth scrolling across different refresh rates.

Clay_GetScrollOffset

```
Clay_Vector2 Clay_GetScrollOffset()
```

Returns the internally stored scroll offset for the currently open element.

Generally intended for use with [clip elements](#) and the `.childOffset` field to create scrolling containers.

See [Scrolling Elements](#) for more details.

```
// Create a horizontally scrolling container
CLAY(CLAY_ID("ScrollContainer"), {
    .clip = { .horizontal = true, .childOffset = Clay_GetScrollOffset() }
})
```

Clay_BeginLayout

```
void Clay_BeginLayout()
```

Prepares clay to calculate a new layout. Called each frame / layout **before** any of the [Element Macros](#).

Clay_EndLayout

```
Clay_RenderCommandArray Clay_EndLayout()
```

Ends declaration of element macros and calculates the results of the current layout. Renders a [Clay_RenderCommandArray](#) containing the results of the layout calculation.

Clay_Hovered

```
bool Clay_Hovered()
```

Called **during** layout declaration, and returns `true` if the pointer position previously set with `Clay_SetPointerState` is inside the bounding box of the currently open element. Note: this is based on the element's position from the **last** frame.

Clay_OnHover

```
void Clay_OnHover(void (*onHoverFunction)(Clay_ElementId elementId, Clay_PointerData pointerData, void *userData), void *userData)
```

Called **during** layout declaration, this function allows you to attach a function pointer to the currently open element that will be called once per layout if the pointer position previously set with `Clay_SetPointerState` is inside the bounding box of the currently open element. See [Clay_PointerData](#) for more information on the `pointerData` argument.

```
void HandleButtonInteraction(Clay_ElementId elementId, Clay_PointerData pointerData, void *userData) {
    ButtonData *buttonData = (ButtonData *)userData;
    // Pointer state allows you to detect mouse down / hold / release
    if (pointerData.state == CLAY_POINTER_DATA_PRESSED_THIS_FRAME) {
        // Do some click handling
        NavigateTo(buttonData->link);
    }
}

ButtonData linkButton = (ButtonData) { .link = "https://github.com/nicbarker/clay" };

// HandleButtonInteraction will be called for each frame the mouse / pointer / touch is inside the button
// boundaries
CLAY(CLAY_ID("Button"), { .layout = { .padding = CLAY_PADDING_ALL(8) } }) {
```

```
Clay_OnHover(HandleButtonInteraction, &buttonData);
CLAY_TEXT(CLAY_STRING("Click me!"), &headerTextConfig);
}
```

Clay_PointerOver

```
bool Clay_PointerOver(Clay_ElementId id)
```

Returns `true` if the pointer position previously set with `Clay_SetPointerState` is inside the bounding box of the layout element with the provided `id`. Note: this is based on the element's position from the `last` frame. If frame-accurate pointer overlap detection is required, perhaps in the case of significant change in UI layout between frames, you can simply run your layout code twice that frame. The second call to `Clay_PointerOver` will be frame-accurate.

Clay_GetScrollContainerData

```
Clay_ScrollContainerData Clay_GetScrollContainerData(Clay_ElementId id)
```

Returns `Clay_ScrollContainerData` for the scroll container matching the provided ID. This function allows imperative manipulation of scroll position, allowing you to build things such as scroll bars, buttons that "jump" to somewhere in a scroll container, etc.

Clay_GetElementData

```
Clay_ElementData Clay_GetElementData(Clay_ElementId id)
```

Returns `Clay_ElementData` for the element matching the provided ID.
Used to retrieve information about elements such as their final calculated bounding box.

Clay_GetElementId

```
Clay_ElementId Clay_GetElementId(Clay_String idString)
```

Returns a [Clay_ElementId](#) for the provided id string, used for querying element info such as mouseover state, scroll container data, etc.

Element Macros

CLAY()

Usage

```
CLAY(...configuration) { ...children }
```

Lifecycle

```
Clay_BeginLayout() -> CLAY() -> Clay_EndLayout()
```

Notes

CLAY opens a generic empty container, that is configurable and supports nested children.

CLAY requires a parameter, so if you want to create an element without any configuration, use `CLAY(0)`.

Examples

```
// Define an element with 16px of x and y padding
CLAY(CLAY_ID("Outer"), { .layout = { .padding = CLAY_PADDING_ALL(16) } }) {
    // A nested child element
    CLAY(CLAY_ID("SideBar"), { .layout = { .layoutDirection = CLAY_TOP_TO_BOTTOM, .childGap = 16 } }) {
        // Children laid out top to bottom with a 16 px gap between them
    }
    // A vertical scrolling container with a colored background
    CLAY(CLAY_ID("ScrollContainer"), {
        .layout = { .layoutDirection = CLAY_TOP_TO_BOTTOM, .childGap = 16 },
        .backgroundColor = { 200, 200, 100, 255 },
        .cornerRadius = CLAY_CORNER_RADIUS(10),
        .clip = { .vertical = true, .childOffset = Clay_GetScrollOffset() }
    }) {
        // child elements
    }
}
```

CLAY_AUTO_ID()

A version of the core [CLAY\(\)](#) element creation macro that generates an ID automatically instead of requiring it as the first argument.

Note that under the hood this ID is generated in the same way as [CLAY_ID_LOCAL\(\)](#), which is based on the element's position in the hierarchy, and may change between layout calls if elements are added / removed from the hierarchy before the element is defined. As a result, for transitions & retained mode backends to work correctly, IDs should be specified.

```
// Note that CLAY_AUTO_ID only takes one argument: the configuration
CLAY_AUTO_ID({ .layout = { .padding = CLAY_PADDING_ALL(16) } }) {
    // A nested child element
    CLAY_AUTO_ID({ .layout = { .layoutDirection = CLAY_TOP_TO_BOTTOM, .childGap = 16 } }) {
        // Children laid out top to bottom with a 16 px gap between them
    }
    // A vertical scrolling container with a colored background
    CLAY_AUTO_ID({
        .layout = { .layoutDirection = CLAY_TOP_TO_BOTTOM, .childGap = 16 },
        .backgroundColor = { 200, 200, 100, 255 },
        .cornerRadius = CLAY_CORNER_RADIUS(10),
        .clip = { .vertical = true, .childOffset = Clay_GetScrollOffset() }
    }) {
        // child elements
    }
}
```

CLAY_TEXT()

Usage

```
CLAY_TEXT(Clay_String textContents, Clay_TextElementConfig *textConfig);
```

Lifecycle

```
Clay_BeginLayout() -> CLAY_TEXT() -> Clay_EndLayout()
```

Notes

TEXT is a measured, auto-wrapped text element. It uses `Clay_TextElementConfig` to configure text specific options.

Note that `Clay_TextElementConfig` uses `uint32_t fontId`. Font ID to font asset mapping is managed in user code and passed to render commands.

Struct API (Pseudocode)

```
// CLAY_TEXT(text, CLAY_TEXT_CONFIG({ .member = value })) supports these options
Clay_TextElementConfig {
    Clay_Color textColor {
        float r; float g; float b; float a;
    };
    uint16_t fontId;
    uint16_t fontSize;
    uint16_t letterSpacing;
    uint16_t lineHeight;
    Clay_TextElementConfigWrapMode wrapMode {
        CLAY_TEXT_WRAP_WORDS (default),
        CLAY_TEXT_WRAP_NEWLINES,
        CLAY_TEXT_WRAP_NONE,
    };
};
```

Fields

`.textColor`

```
CLAY_TEXT_CONFIG(.textColor = {120, 120, 120, 255})
```

Uses [Clay_Color](#). Conventionally accepts `rgba` float values between 0 and 255, but interpretation is left up to the renderer and does not affect layout.

`.fontId`

```
CLAY_TEXT_CONFIG(.fontId = FONT_ID_LATO)
```

It's up to the user to load fonts and create a mapping from `fontId` to a font that can be measured and rendered.

`.fontSize`

```
CLAY_TEXT_CONFIG(.fontSize = 16)
```

Font size is generally thought of as `x pixels tall`, but interpretation is left up to the user & renderer.

`.letterSpacing`

```
CLAY_TEXT_CONFIG(.letterSpacing = 1)
```

`.letterSpacing` results in **horizontal** white space between individual rendered characters.

`.lineHeight`

```
CLAY_TEXT_CONFIG(.lineHeight = 20)
```

`.lineHeight` - when non zero - forcibly sets the `height` of each wrapped line of text to `.lineheight` pixels tall. Will affect the layout of both parents and siblings. A value of `0` will use the measured height of the font.

`.wrapMode`

```
CLAY_TEXT_CONFIG(.wrapMode = CLAY_TEXT_WRAP_NONE)
```

`.wrapMode` specifies under what conditions text should **wrap**.

Available options are:

- `CLAY_TEXT_WRAP_WORDS` (default) - Text will wrap on whitespace characters as container width shrinks, preserving whole words.
 - `CLAY_TEXT_WRAP_NEWLINES` - will only wrap when encountering newline characters.
 - `CLAY_TEXT_WRAP_NONE` - Text will never wrap even if its container is compressed beyond the text measured width.
-

Examples

```
// Define a font somewhere in your code
const uint32_t FONT_ID_LATO = 3;
// ..
CLAY_TEXT(CLAY_STRING("John Smith"), CLAY_TEXT_CONFIG({ .fontId = FONT_ID_LATO, .fontSize = 24, .textColor = {255,
0, 0, 255} }));
// Rendering example
Font fontToUse = LoadedFonts[renderCommand->renderData.text->fontId];
```

Rendering

Element is subject to [culling](#). Otherwise, multiple `Clay_RenderCommand` s with `commandType = CLAY_RENDER_COMMAND_TYPE_TEXT` may be created, one for each wrapped line of text.

`Clay_RenderCommand.textContent` will be populated with a `Clay_String` *slice* of the original string passed in (i.e. wrapping doesn't reallocate, it just returns a `Clay_String` pointing to the start of the new line with a `length`)

CLAY_ID

```
Clay_ElementId CLAY_ID(StringLiteral idString)
```

`CLAY_ID()` is used to generate and attach a `Clay_ElementId` to a layout element during declaration.

Note this macro only works with String literals and won't compile if used with a `char*` variable. To use a heap allocated `char*` string as an ID, use `CLAY_SID`.

To regenerate the same ID outside of layout declaration when using utility functions such as [Clay_PointerOver](#), use the [Clay_GetElementId](#) function.

Examples

```
// Tag a button with the Id "Button"
CLAY(CLAY_ID("Button"), {
    .layout = { .layoutDirection = CLAY_TOP_TO_BOTTOM, .sizing = { .width = CLAY_SIZING_GROW(0) }, .padding =
CLAY_PADDING_ALL(16), .childGap = 16 }
}) {
    // ...children
}

// Later on outside of layout code
bool buttonIsHovered = Clay_IsPointerOver(Clay_GetElementId("Button"));
if (buttonIsHovered && leftMouseButtonPressed) {
    // ... do some click handling
}
```

CLAY_SID()

```
Clay_ElementId CLAY_SID(Clay_String idString)
```

A version of [CLAY_ID](#) that can be used with heap allocated `char *` data. The underlying `char` data will not be copied internally and should live until at least the next frame.

CLAY_IDI()

```
Clay_ElementId CLAY_IDI(StringLiteral idString, int32_t index)
```

An offset version of [CLAY_ID](#). Generates a [Clay_ElementId](#) string id from the provided `char *label`, combined with the `int index`.

Used for generating ids for sequential elements (such as in a `for` loop) without having to construct dynamic strings at runtime.

Note this macro only works with String literals and won't compile if used with a `char*` variable. To use a heap allocated `char*` string as an ID, use [CLAY_SIDI](#).

CLAY_SIDI()

```
Clay_ElementId CLAY_SIDI(Clay_String idString, int32_t index)
```

A version of [CLAY_IDI](#) that can be used with heap allocated `char *` data. The underlying `char` data will not be copied internally and should live until at least the next frame.

CLAY_ID_LOCAL()

Usage

```
Clay_ElementId CLAY_ID_LOCAL(String_LITERAL idString)
```

Lifecycle

```
Clay_BeginLayout() -> CLAY( -> CLAY_ID_LOCAL() -> ) -> Clay_EndLayout()
```

Notes

[CLAY_ID_LOCAL\(\)](#) is used to generate and attach a [Clay_ElementId](#) to a layout element during declaration.

Unlike [CLAY_ID](#) which needs to be globally unique, a local ID is based on the ID of it's parent and only needs to be unique among its siblings.

As a result, local id is suitable for use in reusable components and loops.

Note this macro only works with String literals and won't compile if used with a `char*` variable. To use a heap allocated `char*` string as an ID, use [CLAY_SID_LOCAL](#).

Examples

```

void RenderHeaderButton(ButtonData button) {
    CLAY({
        .id = CLAY_ID_LOCAL("HeaderButton"),
        .layout = { .layoutDirection = CLAY_TOP_TO_BOTTOM, .sizing = { .width = CLAY_SIZING_GROW(0) }, .padding =
CLAY_PADDING_ALL(16), .childGap = 16 }
    }) {
        // ...children
    }
}

for (int i = 0; i < headerButtons.length; i++) {
    RenderHeaderButton(headerButtons.items[i]);
}

```

CLAY_SID_LOCAL()

```
Clay_ElementId CLAY_SID_LOCAL(Clay_String idString)
```

A version of [CLAY_ID_LOCAL](#) that can be used with heap allocated `char *` data. The underlying `char` data will not be copied internally and should live until at least the next frame.

CLAY_IDI_LOCAL()

```
Clay_ElementId CLAY_IDI_LOCAL(String_LITERAL idString, int32_t index)
```

An offset version of [CLAY_ID_LOCAL](#). Generates a [Clay_ElementId](#) string id from the provided `char *label`, combined with the `int` `index`.

Used for generating ids for sequential elements (such as in a `for` loop) without having to construct dynamic strings at runtime.

Note this macro only works with String literals and won't compile if used with a `char*` variable. To use a heap allocated `char*` string as an ID, use [CLAY_SIDI_LOCAL](#).

CLAY_SIDI_LOCAL()

```
Clay_ElementId CLAY_SIDI_LOCAL(Clay_String idString, int32_t index)
```

A version of [CLAY_IDI_LOCAL](#) that can be used with heap allocated `char *` data. The underlying `char` data will not be copied internally and should live until at least the next frame.

Data Structures & Definitions

Clay_ElementDeclaration

The [Clay_ElementDeclaration](#) struct is the only argument to the `CLAY()` macro and provides configuration options for layout elements.

```
typedef struct {  
    Clay_ElementId id;  
    Clay_LayoutConfig layout;  
    Clay_Color backgroundColor;  
    Clay_CornerRadius cornerRadius;  
    Clay_AspectRatioElementConfig aspectRatio;  
    Clay_ImageElementConfig image;  
    Clay_FloatingElementConfig floating;  
    Clay_CustomElementConfig custom;  
    Clay_ClipElementConfig clip;  
    Clay_BorderElementConfig border;  
    void *userData;  
} Clay_ElementDeclaration;
```

Fields

.layout - Clay_LayoutConfig

```
CLAY(CLAY_ID("Element"), { .layout = { .padding = { 16, 16, 12, 12 }, .layoutDirection = CLAY_TOP_TO_BOTTOM } })
```

Uses [Clay_LayoutConfig](#). Controls various settings related to *layout*, which can be thought of as "the size and position of this element and its children".

.backgroundColor - Clay_Color

```
CLAY(CLAY_ID("Element"), { .backgroundColor = {120, 120, 120, 255} } })
```

Uses [Clay_Color](#). Conventionally accepts `rgba` float values between 0 and 255, but interpretation is left up to the renderer and does not affect layout.

.cornerRadius - float

```
CLAY(CLAY_ID("Element"), { .cornerRadius = { .topLeft = 16, .topRight = 16, .bottomLeft = 16, .bottomRight = 16 } })
```

Defines the radius in pixels for the arc of rectangle corners (`0` is square, `rectangle.width / 2` is circular).

Note that the `CLAY_CORNER_RADIUS(radius)` function-like macro is available to provide short hand for setting all four corner radii to the same value. e.g. `CLAY_BORDER({ .cornerRadius = CLAY_CORNER_RADIUS(10) })`

.aspectRatio - Clay_AspectRatioElementConfig

```
CLAY(CLAY_ID("Element"), { .aspectRatio = 1 })
```

Uses [Clay_AspectRatioElementConfig](#). Configures the element as an aspect ratio scaling element. Especially useful for rendering images, but can also be used to enforce a fixed width / height ratio of other elements.

.image - Clay_ImageElementConfig

```
CLAY(CLAY_ID("Element"), { .image = { .imageData = &myImage } })
```

Uses [Clay_ImageElementConfig](#). Configures the element as an image element. Causes a render command with type `IMAGE` to be emitted.

`.floating` - `Clay_FloatingElementConfig`

```
CLAY(CLAY_ID("Element"), { .floating = { .attachTo = CLAY_ATTACH_TO_PARENT } })
```

Uses [Clay_FloatingElementConfig](#). Configures the element as an floating element, which allows it to stack "in front" and "on top" of other elements without affecting sibling or parent size or position.

`.custom` - `Clay_CustomElementConfig`

```
CLAY(CLAY_ID("Element"), { .custom = { .customData = &my3DModel } })
```

Uses [Clay_CustomElementConfig](#). Configures the element as a custom element, which allows you to pass custom data through to the renderer. Causes a render command with type `CUSTOM` to be emitted.

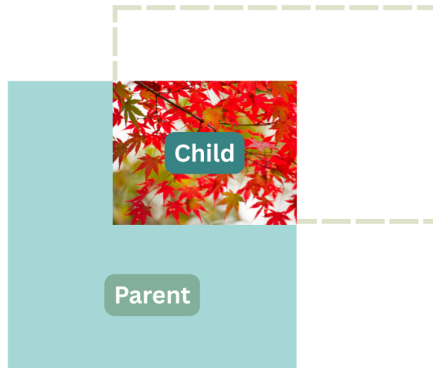
`.clip` - `Clay_ClipElementConfig`

```
CLAY(CLAY_ID("Element"), { .clip = { .vertical = true, .childOffset = Clay_GetScrollOffset() } })
```

Uses [Clay_ClipElementConfig](#). Configures the element as a clip element, which causes child elements to be clipped / masked if they overflow, and together with the functions listed in [Scrolling Elements](#) enables scrolling of child contents.



Clipping **Off**



Clipping **On**

.border - `Clay_BorderElementConfig`

```
CLAY(CLAY_ID("Element"), { .border = { .width = { .left = 5 }, .color = COLOR_BLUE } })
```

Uses [Clay_BorderElementConfig](#). Configures the element as a border element, which instructs the renderer to draw coloured border lines along the perimeter of this element's bounding box. Causes a render command with type `BORDER` to be emitted.

.userData - `void *`

```
CLAY(CLAY_ID("Element"), { .userData = &extraData })
```

Transparently passes a pointer through to the corresponding [Clay_RenderCommandss](#) generated by this element.

Examples


```

// Declare a reusable rectangle config, with a purple color and 10px rounded corners
Clay_RectangleElementConfig rectangleConfig = (Clay_RectangleElementConfig) { .color = { 200, 200, 100, 255 },
    .cornerRadius = CLAY_CORNER_RADIUS(10) };
// Declare a rectangle element using a reusable config
CLAY(CLAY_ID("Box"), rectangleConfig) {}
// Declare a rectangle element using an inline config
CLAY(CLAY_ID("BoxInline"), { .color = { 200, 200, 100, 255 }, .cornerRadius = CLAY_CORNER_RADIUS(10) })) {
    // child elements
}
// Declare a scrolling container with a colored background
CLAY(CLAY_ID("ScrollingContainer"), {
    .backgroundColor = { 200, 200, 100, 255 },
    .cornerRadius = CLAY_CORNER_RADIUS(10)
    .clip = { .vertical = true, .childOffset = Clay_GetScrollOffset() }
} {
    // child elements
}

```

Element is subject to [culling](#). Otherwise, a single `Clay_RenderCommand`s with `commandType = CLAY_RENDER_COMMAND_TYPE_RECTANGLE` will be created, with `renderCommand->elementConfig.rectangleElementConfig` containing a pointer to the element's `Clay_RectangleElementConfig`.

Clay_LayoutConfig

`Clay_LayoutConfig` is used for configuring *layout* options (i.e. options that affect the final position and size of an element, its parents, siblings, and children)

Struct API (Pseudocode)

```

// CLAY({ .layout = { ...fields } }) supports these options
Clay_LayoutConfig {
    Clay_LayoutDirection layoutDirection = CLAY_LEFT_TO_RIGHT (default) | CLAY_TOP_TO_BOTTOM;
    Clay_Padding padding {
        u16 left; u16 right; u16 top; u16 bottom;
    };
    uint16_t childGap;
}

```

```

    Clay_ChildAlignment childAlignment {
        .x = CLAY_ALIGN_X_LEFT (default) | CLAY_ALIGN_X_CENTER | CLAY_ALIGN_X_RIGHT;
        .y = CLAY_ALIGN_Y_TOP (default) | CLAY_ALIGN_Y_CENTER | CLAY_ALIGN_Y_BOTTOM;
    };
    Clay_Sizing sizing { // Recommended to use the provided macros here - see #sizing for more in depth
explanation
        .width = CLAY_SIZING_FIT(float min, float max) (default) | CLAY_SIZING_GROW(float min, float max) |
CLAY_SIZING_FIXED(float width) | CLAY_SIZING_PERCENT(float percent)
        .height = CLAY_SIZING_FIT(float min, float max) (default) | CLAY_SIZING_GROW(float min, float max) |
CLAY_SIZING_FIXED(float height) | CLAY_SIZING_PERCENT(float percent)
    }; // See CLAY_SIZING_GROW() etc for more details
};

```

Fields

.layoutDirection - Clay_LayoutDirection

```
CLAY(CLAY_ID("Element"), { .layout = { .layoutDirection = CLAY_TOP_TO_BOTTOM } })
```

Controls the axis / direction in which child elements are laid out. Available options are `CLAY_LEFT_TO_RIGHT` (default) and `CLAY_TOP_TO_BOTTOM`.

Did you know that "left to right" and "top to bottom" both have 13 letters?

```
.layoutDirection = CLAY_TOP_TO_BOTTOM
```



```
.layoutDirection = CLAY_LEFT_TO_RIGHT
```

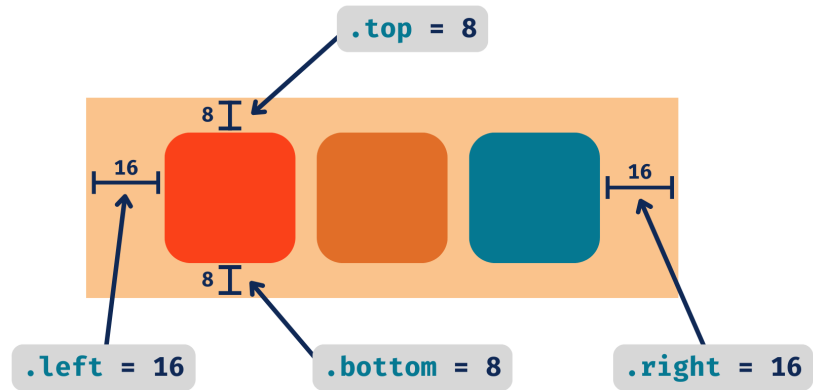


```
.padding - Clay_Padding
```

```
CLAY(CLAY_ID("Element"), { .layout = { .padding = { .left = 16, .right = 16, .top = 8, .bottom = 8 } } })
```

Controls white-space "padding" around the **outside** of child elements.

```
.padding = {16, 16, 8, 8}
```

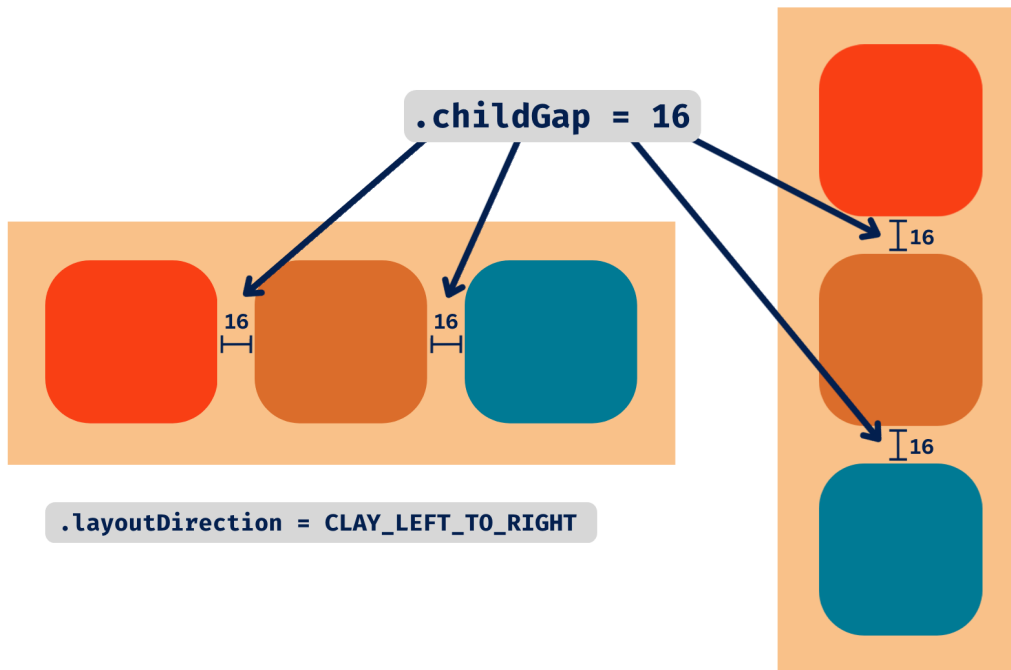


```
.childGap - uint16_t
```

```
CLAY(CLAY_ID("Element"), { .layout = { .childGap = 16 } })
```

Controls the white-space **between** child elements as they are laid out. When `.layoutDirection` is `CLAY_LEFT_TO_RIGHT` (default), this will be horizontal space, whereas for `CLAY_TOP_TO_BOTTOM` it will be vertical space.

`.layoutDirection = CLAY_TOP_TO_BOTTOM`



`.childAlignment` - `Clay_ChildAlignment`

```
CLAY(CLAY_ID("Element"), { .layout = { .childAlignment = { .x = CLAY_ALIGN_X_LEFT, .y = CLAY_ALIGN_Y_CENTER } } })
```

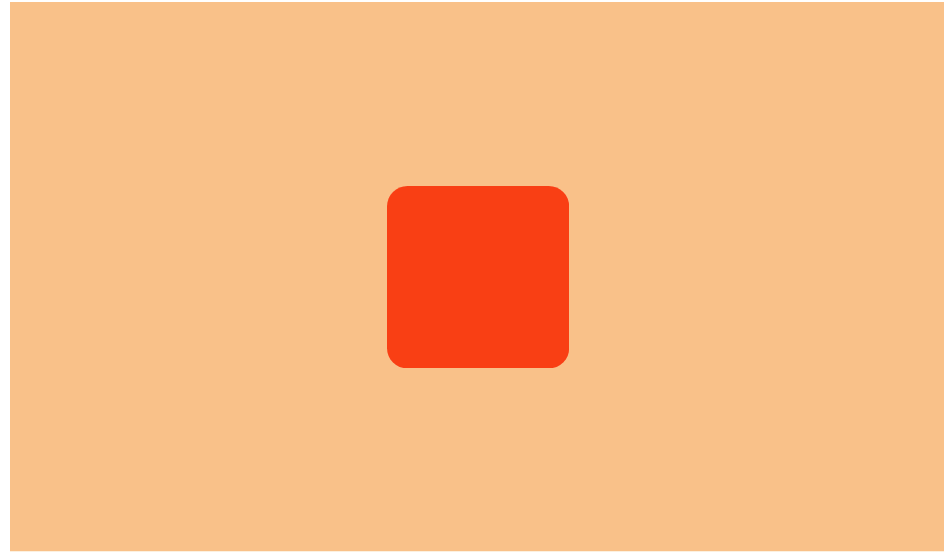
Controls the alignment of children relative to the height and width of the parent container. Available options are:

```
.x = CLAY_ALIGN_X_LEFT (default) | CLAY_ALIGN_X_CENTER | CLAY_ALIGN_X_RIGHT;  
.y = CLAY_ALIGN_Y_TOP (default) | CLAY_ALIGN_Y_CENTER | CLAY_ALIGN_Y_BOTTOM;
```

```
{ .x = CLAY_ALIGN_X_LEFT, .y = CLAY_ALIGN_Y_CENTER }
```



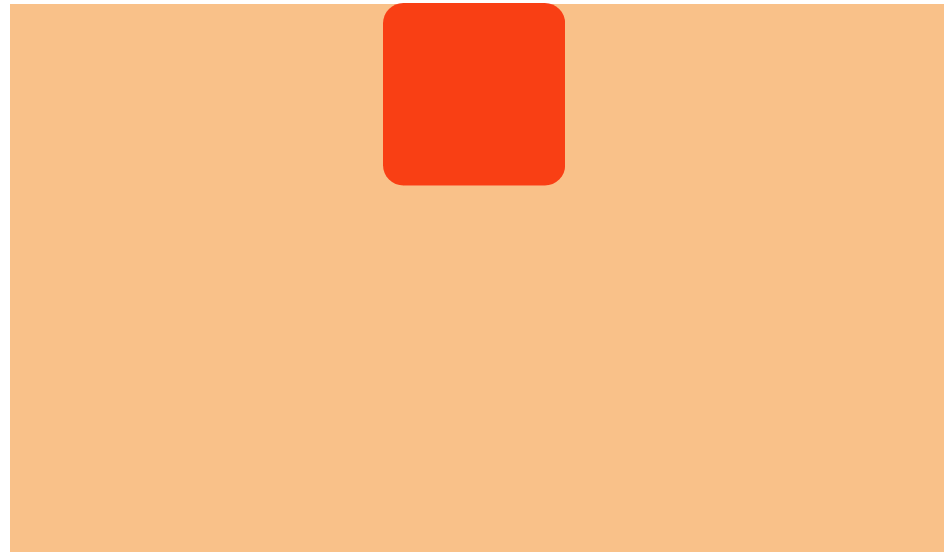
```
{ .x = CLAY_ALIGN_X_CENTER, .y = CLAY_ALIGN_Y_CENTER }
```



```
{ .x = CLAY_ALIGN_X_RIGHT, .y = CLAY_ALIGN_Y_BOTTOM }
```



```
{ .x = CLAY_ALIGN_X_CENTER, .y = CLAY_ALIGN_Y_TOP }
```



.sizing - Clay_Sizing

```
CLAY(CLAY_ID("Element"), { .layout = { .sizing = { .width = CLAY_SIZING_FIXED(300), .height =  
CLAY_SIZING_PERCENT(0.5) } } })
```

Controls how final width and height of element are calculated. The same configurations are available for both the `.width` and `.height` axis. There are several options:

- `CLAY_SIZING_FIT(float min, float max) (default)` - The element will be sized to fit its children (plus padding and gaps), up to `max`. If `max` is left unspecified, it will default to `FLOAT_MAX`. When elements are compressed to fit into a smaller parent, this element will not shrink below `min`.
- `CLAY_SIZING_GROW(float min, float max)` - The element will grow to fill available space in its parent, up to `max`. If `max` is left unspecified, it will default to `FLOAT_MAX`. When elements are compressed to fit into a smaller parent, this element will not shrink below `min`.
- `CLAY_SIZING_FIXED(float fixed)` - The final size will always be exactly the provided `fixed` value. Shorthand for `CLAY_SIZING_FIT(fixed, fixed)`
- `CLAY_SIZING_PERCENT(float percent)` - Final size will be a percentage of parent size, minus padding and child gaps. `percent` is assumed to be a float between `0` and `1`.

```
{ .width = CLAY_SIZING_FIT() }
```



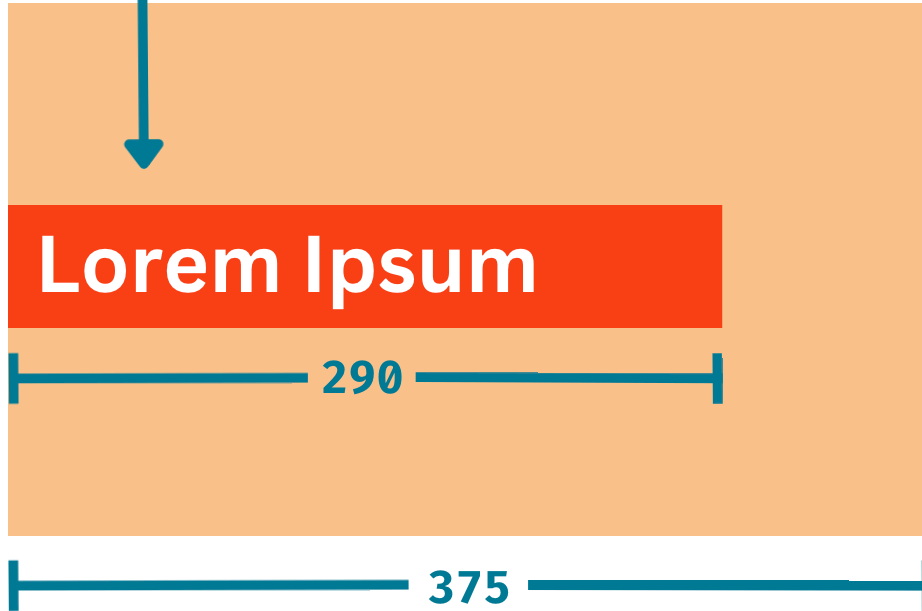
Lorem Ipsum

```
{ .width = CLAY_SIZING_GROW( ) }
```

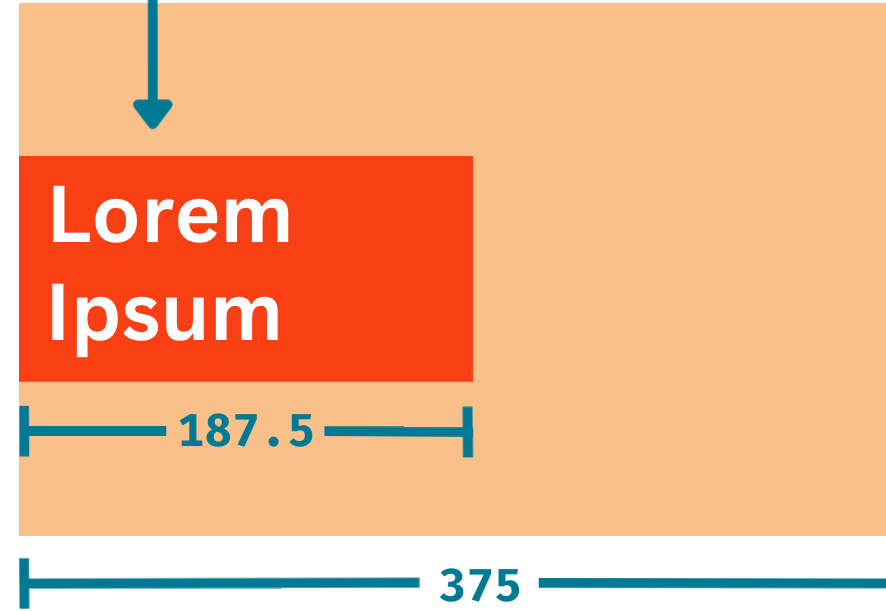


Lorem Ipsum

```
{ .width = CLAY_SIZING_FIXED(290) }
```



```
{ .width = CLAY_SIZING_PERCENT
```



Example Usage

```
CLAY(CLAY_ID("Button"), { .layout = { .layoutDirection = CLAY_TOP_TO_BOTTOM, .sizing = { .width =  
CLAY_SIZING_GROW(0) }, .padding = CLAY_PADDING_ALL(16, .childGap = 16) } }) {  
  // Children will be laid out vertically with 16px of padding around and between  
}
```

Clay_ImageElementConfig

Usage

```
CLAY(CLAY_ID("Element"), { .image = { ...image config } }) {}
```

Clay_ImageElementConfig configures a clay element to render an image as its background.

Struct API (Pseudocode)

```
Clay_ImageElementConfig {  
    void * imageData;  
};
```

Fields

`.imageData` - void *

```
CLAY(CLAY_ID("Image"), { .image = { .imageData = &myImage } }) {}
```

`.imageData` is a generic void pointer that can be used to pass through image data to the renderer.

```
// Load an image somewhere in your code  
YourImage profilePicture = LoadYourImage("profilePicture.png");  
// Note that when rendering, .imageData will be void* type.  
CLAY(CLAY_ID("Image"), { .image = { .imageData = &profilePicture } }) {}
```

Examples

```
// Load an image somewhere in your code  
YourImage profilePicture = LoadYourImage("profilePicture.png");  
// Declare a reusable image config  
Clay_ImageElementConfig imageConfig = (Clay_ImageElementConfig) { .imageData = &profilePicture };  
// Declare an image element using a reusable config  
CLAY(CLAY_ID("Image"), { .image = imageConfig }) {}  
// Declare an image element using an inline config  
CLAY(CLAY_ID("ImageInline"), { .image = { .imageData = &profilePicture }, .aspectRatio = 16.0 / 9.0 }) {}  
// Rendering example  
YourImage *imageToRender = renderCommand->elementConfig.imageElementConfig->imageData;
```

Rendering

Element is subject to [culling](#). Otherwise, a single `Clay_RenderCommand`s with `commandType = CLAY_RENDER_COMMAND_TYPE_IMAGE` will be created. The user will need to access `renderCommand->renderData.image->imageData` to retrieve image data referenced during layout

creation. It's also up to the user to decide how / if they wish to blend `renderCommand->renderData.image->backgroundColor` with the image.

Clay_AspectRatioElementConfig

Usage

```
CLAY(CLAY_ID("Aspect"), { .aspectRatio = 16.0 / 9.0 }) {}
```

Clay_AspectRatioElementConfig configures a clay element to enforce a fixed width / height ratio in its final dimensions. Mostly used for image elements, but can also be used for non image elements.

Struct API (Pseudocode)

```
Clay_AspectRatioElementConfig {  
    float aspectRatio;  
};
```

Fields

`.aspectRatio` - float

```
CLAY(CLAY_ID("Aspect"), { .aspectRatio = { .aspectRatio = 16.0 / 9.0 } }) {}
```

or alternatively, as C will automatically pass the value to the first nested struct field:

```
CLAY(CLAY_ID("Aspect"), { .aspectRatio = 16.0 / 9.0 }) {}
```

Examples

```
// Load an image somewhere in your code  
YourImage profilePicture = LoadYourImage("profilePicture.png");  
// Declare an image element that will grow along the X axis while maintaining its original aspect ratio  
CLAY(CLAY_ID("ProfilePicture"), {  
    .layout = { .width = CLAY_SIZING_GROW() },
```

```
.aspectRatio = profilePicture.width / profilePicture.height,  
.image = { .imageData = &profilePicture },  
}) {}
```

Clay_ImageElementConfig

Usage

```
CLAY(CLAY_ID("Image"), { .image = { ...image config } }) {}
```

Clay_ImageElementConfig configures a clay element to render an image as its background.

Struct API (Pseudocode)

```
Clay_ImageElementConfig {  
    void * imageData;  
};
```

Fields

.imageData - void *

```
CLAY(CLAY_ID("Image"), { .image = { .imageData = &myImage } }) {}
```

.imageData is a generic void pointer that can be used to pass through image data to the renderer.

```
// Load an image somewhere in your code  
YourImage profilePicture = LoadYourImage("profilePicture.png");  
// Note that when rendering, .imageData will be void* type.  
CLAY(CLAY_ID("Image"), { .image = { .imageData = &profilePicture } }) {}
```

Note: for an image to maintain its original aspect ratio when using dynamic scaling, the **.aspectRatio** config option must be used.

Examples

```
// Load an image somewhere in your code
YourImage profilePicture = LoadYourImage("profilePicture.png");
// Declare a reusable image config
Clay_ImageElementConfig imageConfig = (Clay_ImageElementConfig) { .imageData = &profilePicture };
// Declare an image element using a reusable config
CLAY(CLAY_ID("Image"), { .image = imageConfig }) {}
// Declare an image element using an inline config
CLAY(CLAY_ID("ImageInline"), { .image = { .imageData = &profilePicture }, .aspectRatio = 16.0 / 9.0 }) {}
// Rendering example
YourImage *imageToRender = renderCommand->elementConfig.imageElementConfig->imageData;
```

Rendering

Element is subject to [culling](#). Otherwise, a single `Clay_RenderCommand`s with `commandType = CLAY_RENDER_COMMAND_TYPE_IMAGE` will be created. The user will need to access `renderCommand->renderData.image->imageData` to retrieve image data referenced during layout creation. It's also up to the user to decide how / if they wish to blend `renderCommand->renderData.image->backgroundColor` with the image.

Clay_ClipElementConfig

Usage

```
CLAY(CLAY_ID("ScrollBox"), { .clip = { ...clip config } }) {}
```

Notes

`Clay_ClipElementConfig` configures the element as a clipping container, enabling masking of children that extend beyond its boundaries.

Note: In order to process scrolling based on pointer position and mouse wheel or touch interactions, you must call `Clay_SetPointerState()` and `Clay_UpdateScrollContainers()` *before* calling `BeginLayout`.

Struct Definition (Pseudocode)

```
Clay_ClipElementConfig {
    bool horizontal;
```

```
bool vertical;  
};
```

Fields

`.horizontal` - bool

```
CLAY(CLAY_ID("HorizontalScroll"), { .clip = { .horizontal = true } })
```

Enables or disables horizontal clipping for this container element.

`.vertical` - bool

```
CLAY(LAY_ID("VerticalScroll"), { .clip = { .vertical = true } })
```

Enables or disables vertical clipping for this container element.

Rendering

Enabling clip for an element will result in two additional render commands:

- `commandType = CLAY_RENDER_COMMAND_TYPE_SCISSOR_START`, which should create a rectangle mask with its `boundingBox` and is **not** subject to **culling**
- `commandType = CLAY_RENDER_COMMAND_TYPE_SCISSOR_END`, which disables the previous rectangle mask and is **not** subject to **culling**

Examples

```
CLAY(CLAY_ID("ScrollOuter"), { .clip = { .vertical = true } }) {  
    // Create child content with a fixed height of 5000  
    CLAY(CLAY_ID("ScrollInner"), { .layout = { .sizing = { .height = CLAY_SIZING_FIXED(5000) } } }) {}  
}
```

Clay_BorderElementConfig

Usage

```
CLAY(CLAY_ID("Border"), { .border = { ...border config } }) {}
```

Notes

`Clay_BorderElementConfig` adds borders to the edges or between the children of elements. It uses `Clay_BorderElementConfig` to configure border specific options.

Struct Definition (Pseudocode)

```
typedef struct Clay_BorderElementConfig
{
    Clay_Color color {
        float r; float g; float b; float a;
    };
    Clay_BorderWidth width {
        uint16_t left;
        uint16_t right;
        uint16_t top;
        uint16_t bottom;
        uint16_t betweenChildren;
    };
} Clay_BorderElementConfig;
```

Fields

.color - `Clay_Color`

```
CLAY(CLAY_ID("Border"), { .border = { .color = { 255, 0, 0, 255 } } })
```

Uses [Clay_Color](#). Specifies the shared color for all borders configured by this element. Conventionally accepts `rgba` float values between 0 and 255, but interpretation is left up to the renderer and does not affect layout.

`.width` - Clay_BorderWidth

```
CLAY(CLAY_ID("Border"), { .border = { .width = { .left = 2, .right = 10 } } })
```

Indicates to the renderer that a border of `.color` should be draw at the specified edges of the bounding box, **inset and overlapping the box contents by** `.width`.

This means that border configuration does not affect layout, as the width of the border doesn't contribute to the total container width or layout position. Border containers with zero padding will be drawn over the top of child elements.

Note:

`.width.betweenChildren`

```
CLAY(CLAY_ID("Border"), { .border = { .width = { .betweenChildren = 2 } }, .color = COLOR_RED })
```

Configures the width and color of borders to be drawn between children. These borders will be vertical lines if the parent uses `.layoutDirection = CLAY_LEFT_TO_RIGHT` and horizontal lines if the parent uses `CLAY_TOP_TO_BOTTOM`. Unlike `.left`, `.top` etc, this option **will generate additional rectangle render commands representing the borders between children**. As a result, the renderer does not need to specifically implement rendering for these border elements.

Examples

```
// 300x300 container with a 1px red border around all the edges
CLAY(CLAY_ID("OuterBorder"), {
    .layout = { .sizing = { .width = CLAY_SIZING_FIXED(300), .height = CLAY_SIZING_FIXED(300) } },
    .border = { .width = { 1, 1, 1, 1, 0 }, .color = COLOR_RED }
}) {
    // ...
}

// Container with a 3px yellow bottom border
CLAY(CLAY_ID("OuterBorder"), {
    .border = { .width = { .bottom = 3 }, .color = COLOR_YELLOW }
}) {
    // ...
}
```

```

}

// Container with a 5px curved border around the edges, and a 5px blue border between all children laid out top to bottom
CLAY(CLAY_ID("OuterBorder"), {
    .layout = { .layoutDirection = CLAY_TOP_TO_BOTTOM },
    .border = { .width = { 5, 5, 5, 5, 5 }, .color = COLOR_BLUE }
}) {
    // Child
    // -- 5px blue border will be here --
    // Child
    // -- 5px blue border will be here --
    // Child
}

```

Rendering

Element is subject to [culling](#). Otherwise, a single `Clay_RenderCommand` with `commandType = CLAY_RENDER_COMMAND_TYPE_BORDER` representing the container will be created.

Rendering of borders and rounded corners is left up to the user. See the provided [Raylib Renderer](#) for examples of how to draw borders using line and curve primitives.

Clay_FloatingElementConfig

Usage

```
CLAY(CLAY_ID("Floating"), { .floating = { ...floating config } }) {}
```

Notes

Floating Elements defines an element that "floats" above other content. Typical use-cases include tooltips and modals.

Floating containers:

- With the default configuration, attach to the top left corner of their "parent"

- Don't affect the width and height of their parent
- Don't affect the positioning of sibling elements
- Depending on their z-index can appear above or below other elements, partially or completely occluding them
- Apart from positioning, function just like standard elements - including expanding to fit their children, etc.

The easiest mental model to use when thinking about floating containers is that they are a completely separate UI hierarchy, attached to a specific x,y point on their "parent".

Floating elements uses `Clay_FloatingElementConfig` to configure specific options.

Struct Definition (Pseudocode)

```
Clay_FloatingElementConfig {
    Clay_Vector2 offset {
        float x, float y
    };
    Clay_Dimensions expand {
        float width, float height
    };
    uint32_t parentId;
    int16_t zIndex;
    Clay_FloatingAttachPoints attachPoint {
        .element = CLAY_ATTACH_POINT_LEFT_TOP (default) | CLAY_ATTACH_POINT_LEFT_CENTER |
CLAY_ATTACH_POINT_LEFT_BOTTOM | CLAY_ATTACH_POINT_CENTER_TOP | CLAY_ATTACH_POINT_CENTER_CENTER |
CLAY_ATTACH_POINT_CENTER_BOTTOM | CLAY_ATTACH_POINT_RIGHT_TOP | CLAY_ATTACH_POINT_RIGHT_CENTER |
CLAY_ATTACH_POINT_RIGHT_BOTTOM
        .parent = CLAY_ATTACH_POINT_LEFT_TOP (default) | CLAY_ATTACH_POINT_LEFT_CENTER |
CLAY_ATTACH_POINT_LEFT_BOTTOM | CLAY_ATTACH_POINT_CENTER_TOP | CLAY_ATTACH_POINT_CENTER_CENTER |
CLAY_ATTACH_POINT_CENTER_BOTTOM | CLAY_ATTACH_POINT_RIGHT_TOP | CLAY_ATTACH_POINT_RIGHT_CENTER |
CLAY_ATTACH_POINT_RIGHT_BOTTOM
    };
    Clay_FloatingAttachToElement attachTo {
        CLAY_POINTER_CAPTURE_MODE_CAPTURE (default),
        CLAY_POINTER_CAPTURE_MODE_PASSTHROUGH
    };
    Clay_FloatingAttachToElement attachTo {
        CLAY_ATTACH_TO_NONE (default),
        CLAY_ATTACH_TO_PARENT,
```

```
        CLAY_ATTACH_TO_ELEMENT_WITH_ID,  
        CLAY_ATTACH_TO_ROOT,  
    };  
};
```

Fields

.offset - Clay_Vector2

```
CLAY(CLAY_ID("Floating"), { .floating = { .offset = { -24, -24 } } })
```

Used to apply a position offset to the floating container *after* all other layout has been calculated.

.expand - Clay_Dimensions

```
CLAY(CLAY_ID("Floating"), { .floating = { .expand = { 16, 16 } } })
```

Used to expand the width and height of the floating container *before* laying out child elements.

.zIndex - float

```
CLAY(CLAY_ID("Floating"), { .floating = { .zIndex = 1 } })
```

All floating elements (as well as their entire child hierarchies) will be sorted by **.zIndex** order before being converted to render commands. If render commands are drawn in order, elements with higher **.zIndex** values will be drawn on top.

.parentId - uint32_t

```
CLAY(CLAY_ID("Floating"), { .floating = { .parentId = Clay_GetElementId("HeaderButton").id } })
```

By default, floating containers will "attach" to the parent element that they are declared inside. However, there are cases where this limitation could cause significant performance or ergonomics problems. **.parentId** allows you to specify a **CLAY_ID().id** to attach the floating

container to. The parent element with the matching id can be declared anywhere in the hierarchy, it doesn't need to be declared before or after the floating container in particular.

Consider the following case:

```
// Load an image somewhere in your code
CLAY(CLAY_IDI("SidebarButton", 1), { }) {
    // .. some button contents
    if (tooltip.attachedButtonIndex == 1) {
        CLAY(CLAY_ID("OptionTooltip"), { /* floating config... */ })
    }
}
CLAY(CLAY_IDI("SidebarButton", 2), { }) {
    // .. some button contents
    if (tooltip.attachedButtonIndex == 2) {
        CLAY(CLAY_ID("OptionTooltip"), { /* floating config... */ })
    }
}
CLAY(CLAY_IDI("SidebarButton", 3), { }) {
    // .. some button contents
    if (tooltip.attachedButtonIndex == 3) {
        CLAY(CLAY_ID("OptionTooltip"), { /* floating config... */ })
    }
}
CLAY(CLAY_IDI("SidebarButton", 4), { }) {
    // .. some button contents
    if (tooltip.attachedButtonIndex == 4) {
        CLAY(CLAY_ID("OptionTooltip"), { /* floating config... */ })
    }
}
CLAY(CLAY_IDI("SidebarButton", 5), { }) {
    // .. some button contents
    if (tooltip.attachedButtonIndex == 5) {
        CLAY(CLAY_ID("OptionTooltip"), { /* floating config... */ })
    }
}
```

The definition of the above UI is significantly polluted by the need to conditionally render floating tooltips as a child of many possible elements. The alternative, using `parentId`, looks like this:

```
// Load an image somewhere in your code
CLAY(CLAY_IDI("SidebarButton", 1), { }) {
    // .. some button contents
}
CLAY(CLAY_IDI("SidebarButton", 2), { }) {
    // .. some button contents
}
CLAY(CLAY_IDI("SidebarButton", 3), { }) {
    // .. some button contents
}
CLAY(CLAY_IDI("SidebarButton", 4), { }) {
    // .. some button contents
}
CLAY(CLAY_IDI("SidebarButton", 5), { }) {
    // .. some button contents
}

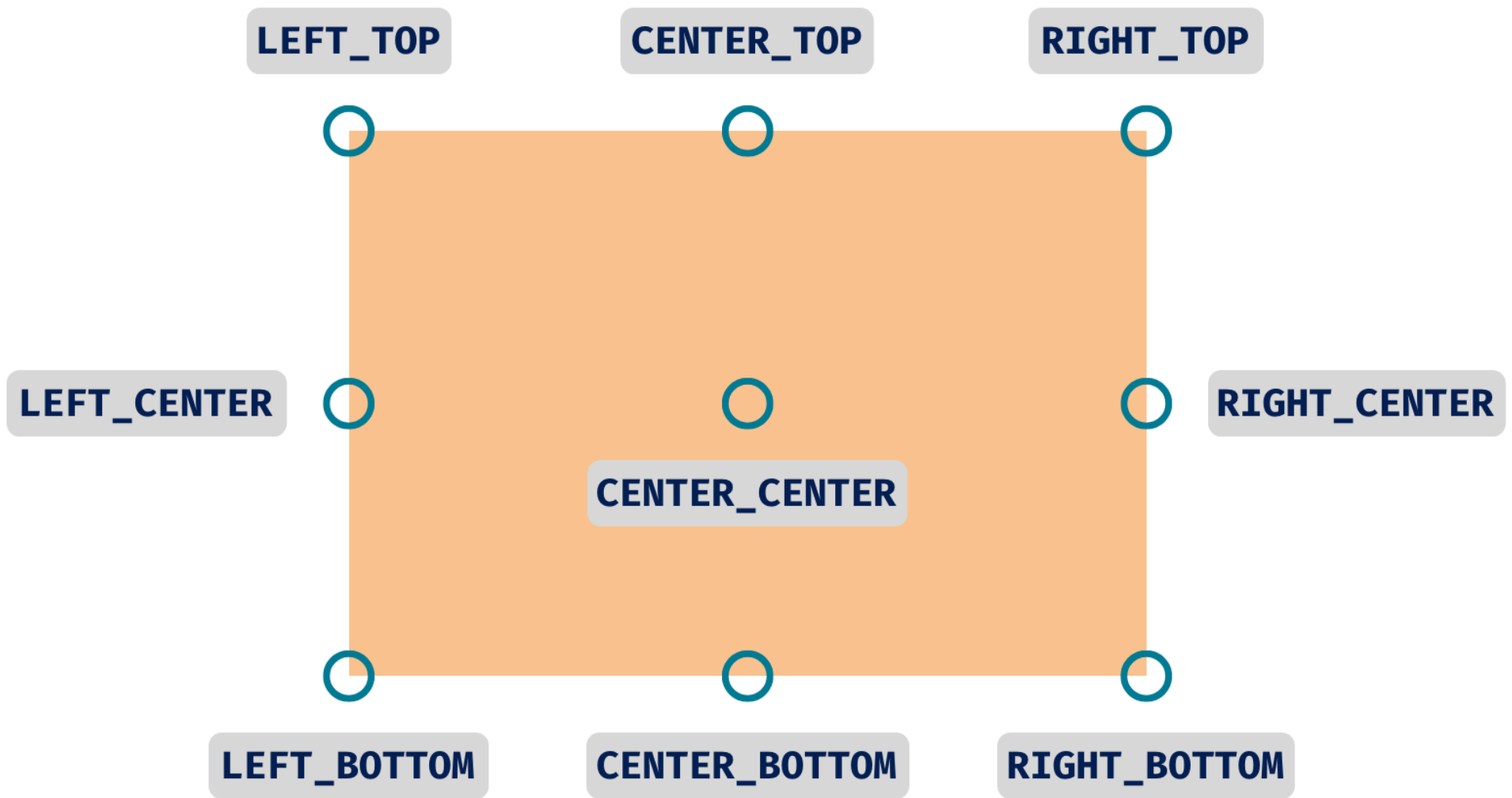
// Any other point in the hierarchy
CLAY(CLAY_ID("OptionTooltip"), { .floating = { .attachTo = CLAY_ATTACH_TO_ELEMENT_ID, .parentId =
CLAY_IDI("SidebarButton", tooltip.attachedButtonIndex).id } }) {
    // Tooltip contents...
}
```

`.attachment` - Clay_FloatingAttachPoints

```
CLAY(CLAY_ID("Floating"), { .floating = { .attachment = { .element = CLAY_ATTACH_POINT_LEFT_CENTER, .parent =
CLAY_ATTACH_POINT_RIGHT_TOP } } }) { }
```

In terms of positioning the floating container, `.attachment` specifies

- The point on the floating container (`.element`)
- The point on the parent element that it "attaches" to (`.parent`)



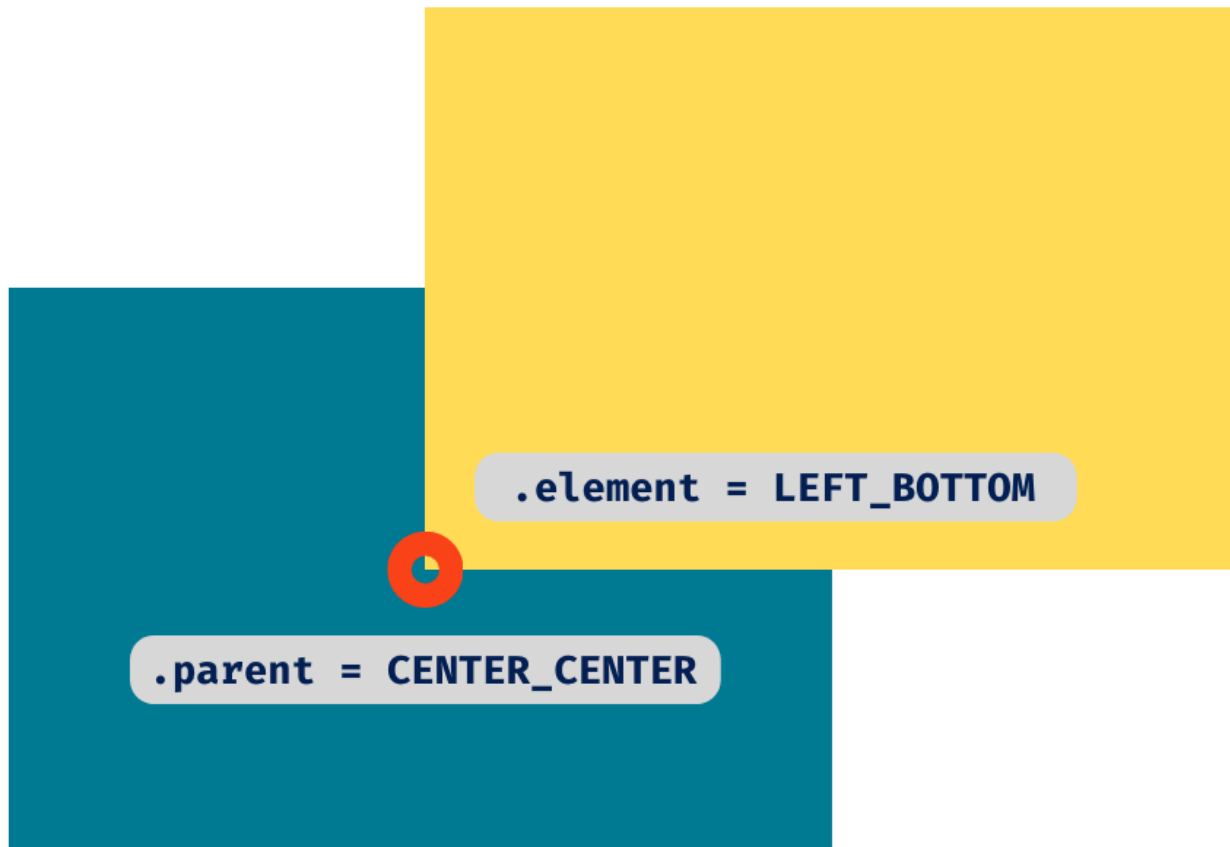
You can mentally visualise this as finding a point on the floating container, then finding a point on the parent, and lining them up over the top of one another.

For example:

"Attach the **LEFT_CENTER** of the floating container to the **RIGHT_TOP** of the parent"

```
CLAY(CLAY_ID("Floating"), { .floating = { .attachment = { .element = CLAY_ATTACH_POINT_LEFT_CENTER, .parent = CLAY_ATTACH_POINT_RIGHT_TOP } } });
```

```
.attachment = {  
  .element = CLAY_ATTACH_POINT_LEFT_BOTTOM,  
  .parent = CLAY_ATTACH_POINT_CENTER_CENTER  
}
```



`.pointerCaptureMode` - `Clay_PointerCaptureMode`

```
CLAY({ .floating = { .pointerCaptureMode = CLAY_POINTER_CAPTURE_MODE_CAPTURE } })
```

Controls whether pointer events like hover and click should pass through to content underneath this floating element, or whether the pointer should be "captured" by this floating element. Defaults to `CLAY_POINTER_CAPTURE_MODE_CAPTURE`.

Examples

```

// Horizontal container with three option buttons
CLAY(CLAY_ID("OptionsList"), { .layout = { childGap = 16 } }) {
    CLAY(CLAY_IDI("Option", 1), { .layout = { padding = CLAY_PADDING_ALL(16)}, .backgroundColor = COLOR_BLUE } })
{
    CLAY_TEXT(CLAY_STRING("Option 1"), CLAY_TEXT_CONFIG());
}
    CLAY(CLAY_IDI("Option", 2), { .layout = { padding = CLAY_PADDING_ALL(16)}, .backgroundColor = COLOR_BLUE } })
{
    CLAY_TEXT(CLAY_STRING("Option 2"), CLAY_TEXT_CONFIG());
    // Floating tooltip will attach above the "Option 2" container and not affect widths or positions of other
elements
    CLAY(CLAY_ID("OptionTooltip"), { .floating = { .zIndex = 1, .attachment = { .element =
CLAY_ATTACH_POINT_CENTER_BOTTOM, .parent = CLAY_ATTACH_POINT_CENTER_TOP } } }) {
        CLAY_TEXT(CLAY_STRING("Most popular!"), CLAY_TEXT_CONFIG());
    }
}
    CLAY(CLAY_IDI("Option", 3), { .layout = { padding = CLAY_PADDING_ALL(16)}, .backgroundColor = COLOR_BLUE } })
{
    CLAY_TEXT(CLAY_STRING("Option 3"), CLAY_TEXT_CONFIG());
}
}

// Floating containers can also be declared elsewhere in a layout, to avoid branching or polluting other UI
for (int i = 0; i < 1000; i++) {
    CLAY(CLAY_IDI("Option", i + 1), { }) {
        // ...
    }
}

// Note the use of "parentId".
// Floating tooltip will attach above the "Option 2" container and not affect widths or positions of other
elements
CLAY(CLAY_ID("OptionTooltip"), { .floating = { .parentId = CLAY_IDI("Option", 2).id, .zIndex = 1, .attachment = {
    .element = CLAY_ATTACH_POINT_CENTER_BOTTOM, .parent = CLAY_ATTACH_POINT_TOP_CENTER } } }) {
    CLAY_TEXT(CLAY_STRING("Most popular!"), CLAY_TEXT_CONFIG());
}

```

When using `.parentId`, the floating container can be declared anywhere after `BeginLayout` and before `EndLayout`. The target element matching the `.parentId` doesn't need to exist when `Clay_FloatingElementConfig` is used.

Rendering

`Clay_FloatingElementConfig` will not generate any specific render commands.

Clay_CustomElementConfig

Usage

```
CLAY(CLAY_ID("Custom"), { .custom = { .customData = &something } }) {}
```

Notes

`Clay_CustomElementConfig` allows the user to pass custom data to the renderer.

Struct Definition (Pseudocode)

```
typedef struct
{
    void * customData;
} Clay_CustomElementConfig;
```

Fields

`.customData` - void *

```
CLAY({ .custom = { .customData = &myCustomData } })
```

`.customData` is a generic void pointer that can be used to pass through custom data to the renderer.

Examples

```
#include "clay.h"
```



```

typedef enum {
    CUSTOM_ELEMENT_TYPE_MODEL,
    CUSTOM_ELEMENT_TYPE_VIDEO
} CustomElementType;

// A rough example of how you could handle laying out 3d models in your UI
typedef struct {
    CustomElementType type;
    union {
        Model model;
        Video video;
        // ...
    };
} CustomElementData;

Model myModel = Load3DModel(filePath);
CustomElement modelElement = (CustomElement) { .type = CUSTOM_ELEMENT_TYPE_MODEL, .model = myModel }

typedef struct {
    void* memory;
    uintptr_t offset;
} Arena;

// During init
Arena frameArena = (Arena) { .memory = malloc(1024) };

// ...
CLAY(0) {
    // Custom elements only take a single pointer, so we need to store the data somewhere
    CustomElementData *modelData = (CustomElementData *) (frameArena.memory + frameArena.offset);
    *modelData = (CustomElementData) { .type = CUSTOM_ELEMENT_TYPE_MODEL, .model = myModel };
    frameArena.offset += sizeof(CustomElementData);
    CLAY(CLAY_ID("3DModelViewer"), { .custom = { .customData = modelData } }) {}
}

// Later during your rendering
switch (renderCommand->commandType) {
    // ...

```

```

    case CLAY_RENDER_COMMAND_TYPE_CUSTOM: {
        // Your extended struct is passed through
        CustomElementData *customElement = renderCommand->config.customElementConfig->customData;
        if (!customElement) continue;
        switch (customElement->type) {
            case CUSTOM_ELEMENT_TYPE_MODEL: {
                // Render your 3d model here
                break;
            }
            case CUSTOM_ELEMENT_TYPE_VIDEO: {
                // Render your video here
                break;
            }
            // ...
        }
        break;
    }
}

```

Rendering

Element is subject to [culling](#). Otherwise, a single `Clay_RenderCommand` with `commandType = CLAY_RENDER_COMMAND_TYPE_CUSTOM` will be created.

Clay_Color

```

typedef struct {
    float r, g, b, a;
} Clay_Color;

```

`Clay_Color` is an RGBA color struct used in Clay's declarations and rendering. By convention the channels are represented as 0-255, but this is left up to the renderer.

Note: when using the debug tools, their internal colors are represented as 0-255.

Clay_String

```
typedef struct {
    bool isStaticallyAllocated;
    int32_t length;
    const char *chars;
} Clay_String;
```

`Clay_String` is a string container that clay uses internally to represent all strings.

Fields

`.isStaticallyAllocated` - `bool`

Whether or not the string is statically allocated, or in other words, whether or not it lives for the entire lifetime of the program.

`.length` - `int32_t`

The number of characters in the string, *not including an optional null terminator*.

`.chars` - `const char *`

A pointer to the contents of the string. This data is not guaranteed to be null terminated, so if you are passing it to code that expects standard null terminated C strings, you will need to copy the data and append a null terminator.

Clay_ElementId

```
typedef struct {
    uint32_t id;
    uint32_t offset;
    uint32_t baseId;
```

```
    Clay_String stringId;  
} Clay_ElementId;
```

Returned by [CLAY_ID](#) and [CLAY_IDI](#), this struct contains a hash id, as well as the source string that was used to generate it.

Fields

`.id` - `uint32_t`

A unique ID derived from the string passed to [CLAY_ID](#) or [CLAY_IDI](#).

`.offset` - `uint32_t`

If this id was generated using [CLAY_IDI](#), `.offset` is the value passed as the second argument. For [CLAY_ID](#), this will always be `0`.

`.baseId` - `uint32_t`

If this id was generated using [CLAY_IDI](#), `.baseId` is the hash of the base string passed, **before it is additionally hashed with** `.offset`. For [CLAY_ID](#), this will always be the same as `.id`.

`.stringId` - `Clay_String`

Stores the original string that was passed in when [CLAY_ID](#) or [CLAY_IDI](#) were called.

Clay_RenderCommandArray

```
typedef struct  
{
```

```
uint32_t capacity;  
uint32_t length;  
Clay_RenderCommand *internalArray;  
} Clay_RenderCommandArray;
```

Returned by [Clay_EndLayout](#), this array contains the [Clay_RenderCommands](#) representing the calculated layout.

Fields

.capacity - uint32_t

Represents the total capacity of the allocated memory in **.internalArray**.

.length - uint32_t

Represents the total number of **Clay_RenderCommand** elements stored consecutively at the address **.internalArray**.

.internalArray - Clay_RenderCommand

An array of [Clay_RenderCommands](#) representing the calculated layout. If there was at least one render command, this array will contain elements from **.internalArray[0]** to **.internalArray[.length - 1]**.

Clay_RenderCommand

```
typedef struct {  
    Clay_BoundingBox boundingBox;  
    Clay_RenderData renderData;  
    uintptr_t userData;  
    uint32_t id;  
    int16_t zIndex;
```

```
    Clay_RenderCommandType commandType;  
} Clay_RenderCommand;
```

Fields

.commandType - Clay_RenderCommandType

An enum indicating how this render command should be handled. Possible values include:

- CLAY_RENDER_COMMAND_TYPE_NONE - Should be ignored by the renderer, and never emitted by clay under normal conditions.
- CLAY_RENDER_COMMAND_TYPE_RECTANGLE - A rectangle should be drawn, configured with `.config.rectangleElementConfig`
- CLAY_RENDER_COMMAND_TYPE_BORDER - A border should be drawn, configured with `.config.borderElementConfig`
- CLAY_RENDER_COMMAND_TYPE_TEXT - Text should be drawn, configured with `.config.textElementConfig`
- CLAY_RENDER_COMMAND_TYPE_IMAGE - An image should be drawn, configured with `.config.imageElementConfig`
- CLAY_RENDER_COMMAND_TYPE_SCISSOR_START - Named after `glScissor`, this indicates that the renderer should begin culling any subsequent pixels that are drawn outside the `.boundingBox` of this render command.
- CLAY_RENDER_COMMAND_TYPE_SCISSOR_END - Only ever appears after a matching `CLAY_RENDER_COMMAND_TYPE_SCISSOR_START` command, and indicates that the scissor has ended.
- CLAY_RENDER_COMMAND_TYPE_CUSTOM - A custom render command controlled by the user, configured with `.config.customElementConfig`

.boundingBox - Clay_BoundingBox

```
typedef struct {  
    float x, y, width, height;  
} Clay_BoundingBox;
```

A rectangle representing the bounding box of this render command, with `.x` and `.y` representing the top left corner of the element.

.id - uint32_t

The id that was originally used with the element macro that created this render command. See [CLAY_ID](#) for details.

`.zIndex` - `int16_t`

The z index of the element, based on what was passed to the root floating configuration that this element is a child of. Higher z indexes should be rendered *on top* of lower z indexes.

`.renderData` - `Clay_RenderData`

```
typedef union {
    Clay_RectangleRenderData rectangle;
    Clay_TextRenderData text;
    Clay_ImageRenderData image;
    Clay_CustomRenderData custom;
    Clay_BorderRenderData border;
} Clay_RenderData;
```

A C union containing various structs, with the type dependent on `.commandType`. Possible values include:

- `config.rectangle` - Used when `.commandType == CLAY_RENDER_COMMAND_TYPE_RECTANGLE`.
- `config.text` - Used when `.commandType == CLAY_RENDER_COMMAND_TYPE_TEXT`. See [Clay_Text](#) for details.
- `config.image` - Used when `.commandType == CLAY_RENDER_COMMAND_TYPE_IMAGE`. See [Clay_Image](#) for details.
- `config.border` - Used when `.commandType == CLAY_RENDER_COMMAND_TYPE_BORDER`. See [Clay_Border](#) for details.
- `config.custom` - Used when `.commandType == CLAY_RENDER_COMMAND_TYPE_CUSTOM`. See [Clay_Custom](#) for details.

Union Structs

```
typedef struct {
    Clay_StringSlice stringContents;
    Clay_Color textColor;
    uint16_t fontId;
    uint16_t fontSize;
```

```
uint16_t letterSpacing;  
uint16_t lineHeight;  
} Clay_TextRenderData;
```

```
typedef struct {  
    Clay_Color backgroundColor;  
    Clay_CornerRadius cornerRadius;  
} Clay_RectangleRenderData;
```

```
typedef struct {  
    Clay_Color backgroundColor;  
    Clay_CornerRadius cornerRadius;  
    void* imageData;  
} Clay_ImageRenderData;
```

```
typedef struct {  
    Clay_Color backgroundColor;  
    Clay_CornerRadius cornerRadius;  
    void* customData;  
} Clay_CustomRenderData;
```

```
typedef struct {  
    Clay_Color color;  
    Clay_CornerRadius cornerRadius;  
    Clay_BorderWidth width;  
} Clay_BorderRenderData;
```

```
typedef union {  
    Clay_RectangleRenderData rectangle;  
    Clay_TextRenderData text;  
    Clay_ImageRenderData image;  
    Clay_CustomRenderData custom;
```



```
    Clay_BorderRenderData border;  
} Clay_RenderData;
```

Clay_ScrollContainerData

```
// Data representing the current internal state of a scrolling element.  
typedef struct {  
    // Note: This is a pointer to the real internal scroll position, mutating it may cause a change in final  
    // layout.  
    // Intended for use with external functionality that modifies scroll position, such as scroll bars or auto  
    // scrolling.  
    Clay_Vector2 *scrollPosition;  
    // The bounding box of the scroll element.  
    Clay_Dimensions scrollContainerDimensions;  
    // The outer dimensions of the inner scroll container content, including the padding of the parent scroll  
    // container.  
    Clay_Dimensions contentDimensions;  
    // The config that was originally passed to the scroll element.  
    Clay_ClipElementConfig config;  
    // Indicates whether an actual scroll container matched the provided ID or if the default struct was returned.  
    bool found;  
} Clay_ScrollContainerData;
```

Fields

.scrollPosition - Clay_Vector2 *

A pointer to the internal scroll position of this scroll container. Mutating it will result in elements inside the scroll container shifting up / down (**.y**) or left / right (**.x**).

.scrollContainerDimensions - Clay_Dimensions

```
typedef struct {  
    float width, height;
```

```
} Clay_Dimensions;
```

Dimensions representing the outer width and height of the scroll container itself.

`.contentDimensions` - `Clay_Dimensions`

```
typedef struct {  
    float width, height;  
} Clay_Dimensions;
```

Dimensions representing the inner width and height of the content *inside* the scroll container. Scrolling is only possible when the `contentDimensions` are larger in at least one dimension than the `scrollContainerDimensions`.

`.config` - `Clay_ClipElementConfig`

The [Clay_ClipElementConfig](#) for the matching scroll container element.

Clay_ElementData

```
// Bounding box and other data for a specific UI element.  
typedef struct {  
    // The rectangle that encloses this UI element, with the position relative to the root of the layout.  
    Clay_BoundingBox boundingBox;  
    // Indicates whether an actual Element matched the provided ID or if the default struct was returned.  
    bool found;  
} Clay_ElementData;
```

Fields

`.boundingBox` - `Clay_BoundingBox`

```
typedef struct {  
    float x, y, width, height;  
} Clay_BoundingBox;
```

A rectangle representing the bounding box of this render command, with `.x` and `.y` representing the top left corner of the element.

`.found` - `bool`

A boolean representing whether or not the ID passed to [Clay_GetElementData](#) matched a valid element or not. In the case that `.found` is `false`, `.boundingBox` will be the default value (zeroed).

Clay_PointerData

```
typedef struct  
{  
    Clay_Vector2 position;  
    Clay_PointerDataInteractionState state;  
} Clay_PointerData;
```

Fields

`.position` - `Clay_Vector2`

A Vector2 containing the current x,y coordinates of the mouse pointer, which were originally passed into [Clay_SetPointerState\(\)](#).

`.state` - `Clay_PointerDataInteractionState`

```
typedef enum
{
    CLAY_POINTER_DATA_PRESSED_THIS_FRAME,
    CLAY_POINTER_DATA_PRESSED,
    CLAY_POINTER_DATA_RELEASED_THIS_FRAME,
    CLAY_POINTER_DATA_RELEASED,
} Clay_PointerDataInteractionState;
```

An enum value representing the current "state" of the pointer interaction. As an example, consider the case where a user is on a desktop computer, moves the mouse pointer over a button, clicks and holds the left mouse button for a short time, then releases it:

- While the mouse pointer is over ("hovering") the button, but no mouse button has been pressed: `CLAY_POINTER_DATA_RELEASED`
- First frame that the user presses the left mouse button: `CLAY_POINTER_DATA_PRESSED_THIS_FRAME`
- All subsequent frames where the user is still holding the left mouse button: `CLAY_POINTER_DATA_PRESSED`
- The single frame where the left mouse button goes from pressed -> released: `CLAY_POINTER_DATA_RELEASED_THIS_FRAME`
- All subsequent frames while the mouse pointer is still over the button: `CLAY_POINTER_DATA_RELEASED`

Clay_ErrorHandler

```
typedef struct
{
    void (*errorHandlerFunction)(Clay_ErrorData errorText);
    uintptr_t userData;
} Clay_ErrorHandler;
```

Fields

.errorHandlerFunction - `void (Clay_ErrorData errorText) {}`

A function pointer to an error handler function, which takes `Clay_ErrorData` as an argument. This function will be called whenever Clay encounters an internal error.

`.userData` - `uintptr_t`

A generic pointer to extra userdata that is transparently passed through from `Clay_Initialize` to Clay's error handler callback. Defaults to NULL.

Clay_ErrorData

```
typedef struct
{
    Clay_ErrorType errorType;
    Clay_String errorText;
    uintptr_t userData;
} Clay_ErrorData;
```

Fields

`.errorType` - `Clay_ErrorType`

```
typedef enum {
    CLAY_ERROR_TYPE_TEXT_MEASUREMENT_FUNCTION_NOT_PROVIDED,
    CLAY_ERROR_TYPE_ARENA_CAPACITY_EXCEEDED,
    CLAY_ERROR_TYPE_ELEMENTS_CAPACITY_EXCEEDED,
    CLAY_ERROR_TYPE_TEXT_MEASUREMENT_CAPACITY_EXCEEDED,
    CLAY_ERROR_TYPE_DUPLICATE_ID,
    CLAY_ERROR_TYPE_FLOATING_CONTAINER_PARENT_NOT_FOUND,
    CLAY_ERROR_TYPE_INTERNAL_ERROR,
} Clay_ErrorType;
```

An enum representing the type of error Clay encountered. It's up to the user to handle on a case by case basis, but as some general guidance:

- `CLAY_ERROR_TYPE_TEXT_MEASUREMENT_FUNCTION_NOT_PROVIDED` - The user is attempting to use `CLAY_TEXT` and either forgot to call `Clay_SetMeasureTextFunction` or accidentally passed a null function pointer.
- `CLAY_ERROR_TYPE_ARENA_CAPACITY_EXCEEDED` - Clay was initialized with an Arena that was too small for the configured `Clay_SetMaxElementCount`. Try using `Clay_MinMemorySize()` to get the exact number of bytes required by the current configuration.

- `CLAY_ERROR_TYPE_ELEMENTS_CAPACITY_EXCEEDED` - The declared UI hierarchy has too many elements for the configured max element count. Use [Clay_SetMaxElementCount](#) to increase the max, then call [Clay_MinMemorySize\(\)](#) again and reinitialize clay's memory with the required size.
 - `CLAY_ERROR_TYPE_ELEMENTS_CAPACITY_EXCEEDED` - The declared UI hierarchy has too much text for the configured text measure cache size. Use [Clay_SetMaxMeasureTextCacheWordCount](#) to increase the max, then call [Clay_MinMemorySize\(\)](#) again and reinitialize clay's memory with the required size.
 - `CLAY_ERROR_TYPE_DUPLICATE_ID` - Two elements in Clays UI Hierarchy have been declared with exactly the same ID. Set a breakpoint in your error handler function for a stack trace back to exactly where this occurred.
 - `CLAY_ERROR_TYPE_FLOATING_CONTAINER_PARENT_NOT_FOUND` - A `CLAY_FLOATING` element was declared with the `.parentId` property, but no element with that ID was found. Set a breakpoint in your error handler function for a stack trace back to exactly where this occurred.
 - `CLAY_ERROR_TYPE_INTERNAL_ERROR` - Clay has encountered an internal logic or memory error. Please report this as a bug with a stack trace to help us fix these!
-

`.errorText` - `Clay_String`

A [Clay_String](#) that provides a human readable description of the error. May change in future and should not be relied on to detect error types.

`.userData` - `uintptr_t`

A generic pointer to extra userdata that is transparently passed through from `Clay_Initialize` to Clay's error handler callback. Defaults to NULL.
